



Разработка API-контрактов на Java Spring

1. Введение

В современных системах клиентские приложения и сервисы часто общаются через HTTP. Без явного контракта команды конфликтуют:

- Фронтенд ждёт поле `fullName`, а бекенд вернул `firstName + lastName` отдельно - приложение заработало с ошибкой.
- Мобильное приложение отправляет PATCH с одним полем, но сервер ожидает все поля (PUT-семантика) – в итоге данные затираются нулями.

Контракт, оформленный как отдельная зависимость, устраняет эти проблемы - и клиент, и сервер разрабатываются на основе одного кода.

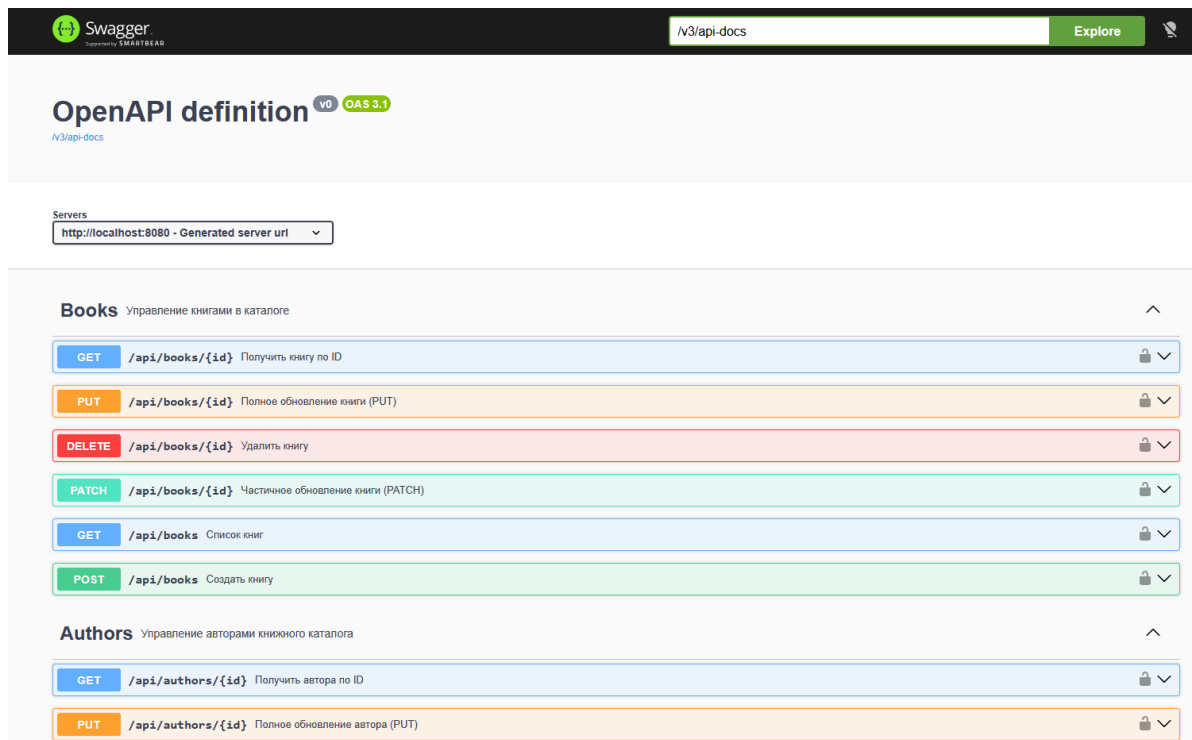
Структура проекта и необходимое ПО

Итак, вам предоставляется пример проекта с контрактом, который описывается в виде аннотаций OpenAPI спецификации - `books-api-contract`.

Требования:

- Java 21
- Maven 3.9+
- IDE с поддержкой Maven (IntelliJ IDEA, VS Code + Extension Pack for Java)

Документация swagger выглядит так:



Но давайте все по порядку!

2. Теория

2.1 Что такое API-контракт и зачем он нужен?

API-контракт — это соглашение между производителем (сервером) и потребителем (клиентом) о том, как именно они общаются: какие URL существуют, какие данные принимаются и возвращаются, какие ошибки возможны.

Контракт в виде отдельной зависимости даёт ключевое преимущество — **compile-time гарантию**: если реализация нарушает контракт (изменили сигнатуру метода или тип поля), проект **не скомпилируется**. Ошибку увидят немедленно, а не на продакшене.

2.2 Пример контракта на примере AuthorApi

AuthorApi — это Java-интерфейс с Spring MVC аннотациями. Реализующий контроллер в demo-rest просто пишет implements AuthorApi и заполняет методы бизнес-логикой.

```
// Контракт
@Tag(name = "Authors")
@RequestMapping(value = "/api/authors", produces =
APPLICATION_JSON_VALUE)
public interface AuthorApi {
```



```
@GetMapping("/{id}")
EntityModel<AuthorResponse> getAuthorById(@PathVariable Long
id);

@PostMapping(consumes = APPLICATION_JSON_VALUE)
@ResponseStatus(HttpStatus.CREATED)
ResponseEntity<EntityModel<AuthorResponse>>
createAuthor(@Valid @RequestBody AuthorRequest request);
}
```

// Реализация

@RestController

public class AuthorController implements AuthorApi {

@Override

```
public EntityModel<AuthorResponse> getAuthorById(Long id) {
    return assembler.toModel(authorService.findById(id));
}
```

}

Контроллер **не повторяет** URL, HTTP-метод, аннотации валидации и документации — всё это уже в интерфейсе.

2.3 Request vs Response

Request — данные, которые клиент отправляет серверу. Используются record'ы — неизменяемые, лаконичные.

```
public record AuthorRequest(
    @NotBlank String firstName,
    @NotBlank String lastName,
    @Email      String email,
    @Past      LocalDate birthDate
) {}
```



Response — данные, которые сервер возвращает клиенту. Поскольку ответы поддерживают HATEOAS-ссылки (наследование от `RepresentationModel`), здесь нужен обычный класс с Lombok.

```
@Getter
@Builder
@EqualsAndHashCode(callSuper = false)
@JsonInclude(JsonInclude.Include.NON_NULL)    // null-поля не
попадают в JSON
@Relation(collectionRelation = "authors", itemRelation =
"author")
public class AuthorResponse extends
RepresentationModel<AuthorResponse> {
    private final Long id;
    private final String firstName;
    private final String lastName;
    private final String fullName;    // firstName + lastName,
вычисляемое
    private final Integer booksCount;
    // ...
}
```

Почему не record?

record — финальный класс, он не может наследовать другой класс.

RepresentationModel нужен для добавления HATEOAS-ссылок (*_links*).

2.4 PUT vs PATCH

Три разных DTO для обновления для точного выражение семантики HTTP:

HTTP-метод	DTO	Смысл
POST	BookRequest	Создать новую книгу, указать автора
PUT /books/{id}	UpdateBookRequest	Полная замена — все поля обязательны



PATCH /books/{id}	PatchBookRequest	Частичное обновление — только то, что меняется
-------------------	------------------	--

Для PATCH все поля nullable. Логика в сервисе: null = "не трогать это поле".

// PUT — обязательные поля

```
public record UpdateBookRequest(  
    @NotBlank String title,    // обязательно  
    @NotBlank String isbn,     // обязательно  
    String description         // необязательно, но при PUT  
                                затрётся null если не передать  
) {}
```

// PATCH — всё необязательно

```
public record PatchBookRequest(  
    String title,              // null = не менять  
    String isbn,                // null = не менять  
    String description         // null = не менять  
) {}
```

2.5 HATEOAS и пагинация

HATEOAS (Hypermedia As The Engine Of Application State) — добавление ссылок к ответу. Клиент не «зашивает» URL, а получает их из ответа сервера.

```
{  
  "id": 1,  
  "firstName": "Лев",  
  "_links": {  
    "self": { "href": "/api/authors/1" },  
    "authors": { "href": "/api/authors" },  
    "books": { "href": "/api/authors/1/books" }  
  }  
}
```

Пагинация — для коллекций используется `PagedModel<EntityModel<T>>`. В контракте это выражается через `PagedResponse<T>`:

```
public record PagedResponse<T>(  
    List<T> content,           // элементы текущей страницы
```



```
        int pageNumber,           // номер страницы (0-based)
        int pageSize,             // размер страницы
        long totalElements,       // всего элементов в базе
        int totalPages,           // всего страниц
        boolean last              // это последняя страница?
    ) {}
```

Запрос с пагинацией выглядит так:

GET /api/authors?page=0&size=20

2.6 Обработка ошибок

Вместо произвольного JSON для ошибок используется стандарт **RFC 7807**, который понимают все клиенты.

```
public record ErrorResponse(
    int status,                // HTTP-код: 404
    String type,               // URI типа ошибки (машиночитаем)
    String title,              // краткое название
    String detail,             // подробности конкретного случая
    String instance,           // URI запроса, вызвавшего ошибку
    Instant timestamp,         // когда произошло
    List<FieldError> fieldErrors // список ошибок полей (для
400)
) {}
```

Пример ответа на 404:

```
{
  "status": 404,
  "type": "https://api.example.com/problems/resource-not-found",
  "title": "Ресурс не найден",
  "detail": "Author with id=99 not found",
  "instance": "/api/authors/99",
  "timestamp": "2026-03-03T14:00:00Z"
}
```

Пример ответа на 400 (ошибка валидации):

```
{
```



```
"status": 400,  
"title": "Ошибка валидации",  
"fieldErrors": [  
    { "field": "isbn", "rejectedValue": "123", "message":  
"Некорректный ISBN" },  
    { "field": "title", "rejectedValue": "", "message": "Название  
не может быть пустым" }  
]  
}
```

2.7 Кастомная валидация: @ValidIsbn

Стандартные аннотации (@NotBlank, @Size, @Email) не умеют проверять ISBN. Создаём собственную.

Шаг 1. Объявить аннотацию:

```
@Documented  
@Constraint(validatedBy = IsbnValidator.class) // ← связываем с  
логикой  
@Target({ElementType.FIELD, ElementType.PARAMETER})  
@Retention(RetentionPolicy.RUNTIME)  
public @interface ValidIsbn {  
    String message() default "Некорректный ISBN. Допустимы ISBN-  
10 и ISBN-13";  
    Class<?>[] groups() default {};  
    Class<? extends Payload>[] payload() default {};  
}
```

Три метода (message, groups, payload) — обязательный контракт Bean Validation API.

Шаг 2. Написать логику:

```
public class IsbnValidator implements  
ConstraintValidator<ValidIsbn, String> {  
  
    private static final Pattern ISBN_10 =  
Pattern.compile("^\\d{9}[\\dX]$");
```



```
private static final Pattern ISBN_13 =
Pattern.compile("^97[89]\\d{10}$");

@Override
public boolean isValid(String value,
ConstraintValidatorContext ctx) {
    if (value == null || value.isBlank()) return true; //
@NotBlank проверит отдельно
    String cleaned = value.replaceAll("[\\s\\-]", ""); //
убираем дефисы
    return ISBN_10.matcher(cleaned).matches()
        || ISBN_13.matcher(cleaned).matches();
}
}
```

Шаг 3. Использовать в DTO:

```
public record BookRequest(
    @NotBlank String title,
    @NotBlank @ValidIsbn String isbn, // ← наш валидатор
    @NotNull Long authorId
) {}
```

2.8 OpenAPI-конфигурация в контракте

BooksApiContractConfig — класс без @Component. Он несёт только аннотации.

```
@OpenAPIDefinition(
    info = @Info(title = "Books API", version = "1.0.0", ...)
    servers = { @Server(url = "http://localhost:8080",
description = "Local") }
)
@SecurityScheme(
    name = BooksApiContractConfig.SECURITY_SCHEME_BEARER,
    type = SecuritySchemeType.HTTP,
    scheme = "bearer",
    bearerFormat = "JWT"
```




```
)  
public final class BooksApiContractConfig {  
    public static final String SECURITY_SCHEME_BEARER =  
"bearerAuth";  
}
```

springdoc-openapi сканирует весь classpath (включая подключённые JAR) и автоматически подхватывает @OpenAPIDefinition. Сервис-реализатор получает готовый Swagger UI, не написав ни строчки про документацию.

В методах API константа используется так:

```
@Operation(security = @SecurityRequirement(name =  
BooksApiContractConfig.SECURITY_SCHEME_BEARER))
```

3. Задания для самостоятельной работы

Задания выполняются в проекте books-api-contract.

Задание 1. Исследование существующего контракта

1. Откройте `books-api-contract/src/main/java/.../endpoints/`. Найдите интерфейсы `AuthorApi.java` и `BookApi.java`.
2. Для каждого метода в `AuthorApi` заполните таблицу:

Метод интерфейса	HTTP-глагол	URL	Тип ответа	Код успеха
<code>getAuthorById</code>	GET	<code>/api/authors/{id}</code>	<code>EntityModel<AuthorResponse></code>	200
<code>createAuthor</code>	...	...	...	...
...				

1. Откройте `AuthorRequest.java` и `PatchAuthorRequest.java`. Выпишите: какие поля есть только в `AuthorRequest` (не в `PatchAuthorRequest`)? С какими аннотациями?
2. Найдите `ErrorResponse.java`. Чем он отличается от `AuthorResponse`? Почему один - record, а другой - обычный класс?



Задание 2. Добавить поле в AuthorRequest

1. Откройте AuthorRequest.java. Добавьте новое поле - номер телефона.
2. Добавьте то же поле в AuthorResponse.java (новое поле phone типа String с @Schema).
3. В PatchAuthorRequest.java добавьте поле phone.
4. Подумайте, где и как мы должны валидировать новое поле?

Задание 3. Добавить новый метод в AuthorApi

В интерфейс AuthorApi.java добавьте метод для поиска авторов по имени.

Задание 4. Создать кастомный валидатор @ValidYear

1. В пакете validation контракта создайте аннотацию @ValidYear. (Год публикации должен быть между 1000 и текущим годом)
2. Создайте YearValidator.java, реализующий ConstraintValidator<ValidYear, Integer>. Логика: год должен быть от 1000 до текущего года включительно (Year.now().getValue()). Значение null — валидно (за это отвечает @NotNull, не ваш валидатор).
3. Примените @ValidYear к полю publishedYear в BookRequest.java.

Задание 5. Добавить новый DTO BookSummaryResponse

Задача создать укороченную версию ответа для книги - без полного описания и без информации об авторе. Такой DTO полезен в списках, где детальная информация избыточна.

1. В пакете dto создайте BookSummaryResponse.
2. Добавьте в BookApi.java метод:

```
@Operation(summary = "Краткий список всех книг (без деталей)")  
@GetMapping("/summary")  
List<EntityModel<BookSummaryResponse>> getAllBooksSummary();
```

3. Сравните ответы GET /api/books и GET /api/books/summary. Выпишите, какие поля присутствуют в полном ответе, но отсутствуют в summary?



Разработка API-контрактов на Java Spring

1. Введение

В современных системах клиентские приложения и сервисы часто общаются через HTTP. Без явного контракта команды конфликтуют:

- Фронтенд ждёт поле `fullName`, а бекенд вернул `firstName + lastName` отдельно - приложение заработало с ошибкой.
- Мобильное приложение отправляет PATCH с одним полем, но сервер ожидает все поля (PUT-семантика) – в итоге данные затираются нулями.

Контракт, оформленный как отдельная зависимость, устраняет эти проблемы - и клиент, и сервер разрабатываются на основе одного кода.

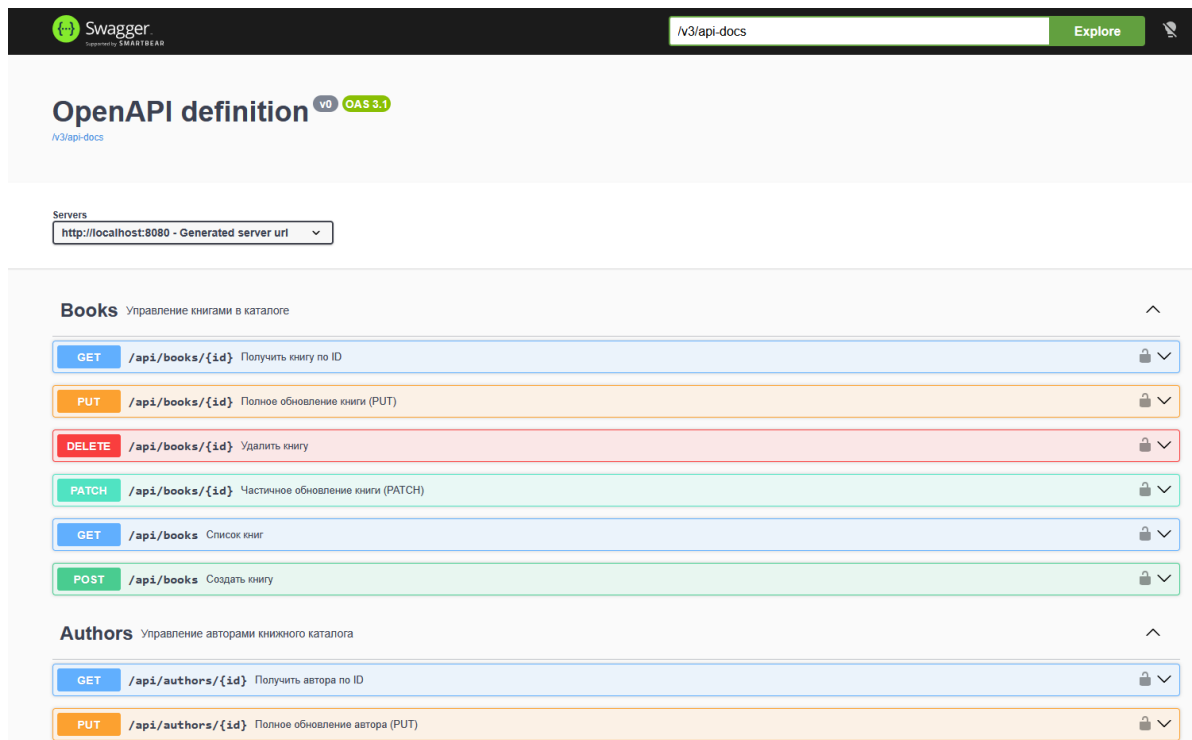
Структура проекта и необходимое ПО

Итак, вам предоставляется пример проекта с контрактом, который описывается в виде аннотаций OpenAPI спецификации - `books-api-contract`.

Требования:

- Java 21
- Maven 3.9+
- IDE с поддержкой Maven (IntelliJ IDEA, VS Code + Extension Pack for Java)

Документация swagger выглядит так:



Но давайте все по порядку!

2. Теория

2.1 Что такое API-контракт и зачем он нужен?

API-контракт — это соглашение между производителем (сервером) и потребителем (клиентом) о том, как именно они общаются: какие URL существуют, какие данные принимаются и возвращаются, какие ошибки возможны.

Контракт в виде отдельной зависимости даёт ключевое преимущество — **compile-time гарантию**: если реализация нарушает контракт (изменили сигнатуру метода или тип поля), проект **не скомпилируется**. Ошибку увидят немедленно, а не на продакшене.

2.2 Пример контракта на примере AuthorApi

AuthorApi — это Java-интерфейс с Spring MVC аннотациями. Реализующий контроллер в demo-rest просто пишет implements AuthorApi и заполняет методы бизнес-логикой.

```
// Контракт
@Tag(name = "Authors")
@RequestMapping(value = "/api/authors", produces =
APPLICATION_JSON_VALUE)
public interface AuthorApi {
```



```
@GetMapping("/{id}")
EntityModel<AuthorResponse> getAuthorById(@PathVariable Long
id);

@PostMapping(consumes = APPLICATION_JSON_VALUE)
@ResponseStatus(HttpStatus.CREATED)
ResponseEntity<EntityModel<AuthorResponse>>
createAuthor(@Valid @RequestBody AuthorRequest request);
}
```

// Реализация

```
@RestController
public class AuthorController implements AuthorApi {

    @Override
    public EntityModel<AuthorResponse> getAuthorById(Long id) {
        return assembler.toModel(authorService.findById(id));
    }
}
```

Контроллер **не повторяет** URL, HTTP-метод, аннотации валидации и документации — всё это уже в интерфейсе.

2.3 Request vs Response

Request — данные, которые клиент отправляет серверу. Используются record'ы — неизменяемые, лаконичные.

```
public record AuthorRequest(
    @NotBlank String firstName,
    @NotBlank String lastName,
    @Email      String email,
    @Past      LocalDate birthDate
) {}
```



Response — данные, которые сервер возвращает клиенту. Поскольку ответы поддерживают HATEOAS-ссылки (наследование от `RepresentationModel`), здесь нужен обычный класс с Lombok.

```
@Getter
@Builder
@EqualsAndHashCode(callSuper = false)
@JsonInclude(JsonInclude.Include.NON_NULL)    // null-поля не
попадают в JSON
@Relation(collectionRelation = "authors", itemRelation =
"author")
public class AuthorResponse extends
RepresentationModel<AuthorResponse> {
    private final Long id;
    private final String firstName;
    private final String lastName;
    private final String fullName;    // firstName + lastName,
вычисляемое
    private final Integer booksCount;
    // ...
}
```

Почему не record?

record — финальный класс, он не может наследовать другой класс.

RepresentationModel нужен для добавления HATEOAS-ссылок (*_links*).

2.4 PUT vs PATCH

Три разных DTO для обновления для точного выражение семантики HTTP:

HTTP-метод	DTO	Смысл
POST	BookRequest	Создать новую книгу, указать автора
PUT /books/{id}	UpdateBookRequest	Полная замена — все поля обязательны



PATCH /books/{id}	PatchBookRequest	Частичное обновление — только то, что меняется
-------------------	------------------	--

Для PATCH все поля nullable. Логика в сервисе: null = "не трогать это поле".

// PUT — обязательные поля

```
public record UpdateBookRequest(  
    @NotBlank String title,    // обязательно  
    @NotBlank String isbn,     // обязательно  
    String description         // необязательно, но при PUT  
                                затрётся null если не передать  
) {}
```

// PATCH — всё необязательно

```
public record PatchBookRequest(  
    String title,              // null = не менять  
    String isbn,                // null = не менять  
    String description         // null = не менять  
) {}
```

2.5 HATEOAS и пагинация

HATEOAS (Hypermedia As The Engine Of Application State) — добавление ссылок к ответу. Клиент не «зашивает» URL, а получает их из ответа сервера.

```
{  
  "id": 1,  
  "firstName": "Лев",  
  "_links": {  
    "self": { "href": "/api/authors/1" },  
    "authors": { "href": "/api/authors" },  
    "books": { "href": "/api/authors/1/books" }  
  }  
}
```

Пагинация — для коллекций используется `PagedModel<EntityModel<T>>`. В контракте это выражается через `PagedResponse<T>`:

```
public record PagedResponse<T>(  
    List<T> content,           // элементы текущей страницы
```



```
        int pageNumber,           // номер страницы (0-based)
        int pageSize,             // размер страницы
        long totalElements,       // всего элементов в базе
        int totalPages,          // всего страниц
        boolean last              // это последняя страница?
    ) {}
```

Запрос с пагинацией выглядит так:

GET /api/authors?page=0&size=20

2.6 Обработка ошибок

Вместо произвольного JSON для ошибок используется стандарт **RFC 7807**, который понимают все клиенты.

```
public record ErrorResponse(
    int status,           // HTTP-код: 404
    String type,          // URI типа ошибки (машиночитаем)
    String title,         // краткое название
    String detail,        // подробности конкретного случая
    String instance,      // URI запроса, вызвавшего ошибку
    Instant timestamp,    // когда произошло
    List<FieldError> fieldErrors // список ошибок полей (для
400)
) {}
```

Пример ответа на 404:

```
{
  "status": 404,
  "type": "https://api.example.com/problems/resource-not-found",
  "title": "Ресурс не найден",
  "detail": "Author with id=99 not found",
  "instance": "/api/authors/99",
  "timestamp": "2026-03-03T14:00:00Z"
}
```

Пример ответа на 400 (ошибка валидации):

```
{
```




```
"status": 400,  
"title": "Ошибка валидации",  
"fieldErrors": [  
    { "field": "isbn", "rejectedValue": "123", "message":  
"Некорректный ISBN" },  
    { "field": "title", "rejectedValue": "", "message": "Название  
не может быть пустым" }  
]  
}
```

2.7 Кастомная валидация: @ValidIsbn

Стандартные аннотации (@NotBlank, @Size, @Email) не умеют проверять ISBN. Создаём собственную.

Шаг 1. Объявить аннотацию:

```
@Documented  
@Constraint(validatedBy = IsbnValidator.class) // ← связываем с  
логикой  
@Target({ElementType.FIELD, ElementType.PARAMETER})  
@Retention(RetentionPolicy.RUNTIME)  
public @interface ValidIsbn {  
    String message() default "Некорректный ISBN. Допустимы ISBN-  
10 и ISBN-13";  
    Class<?>[] groups() default {};  
    Class<? extends Payload>[] payload() default {};  
}
```

Три метода (message, groups, payload) — обязательный контракт Bean Validation API.

Шаг 2. Написать логику:

```
public class IsbnValidator implements  
ConstraintValidator<ValidIsbn, String> {  
  
    private static final Pattern ISBN_10 =  
Pattern.compile("^\\d{9}[\\dX]$");
```



```
private static final Pattern ISBN_13 =
Pattern.compile("^97[89]\\d{10}$");

@Override
public boolean isValid(String value,
ConstraintValidatorContext ctx) {
    if (value == null || value.isBlank()) return true; //
@NotBlank проверит отдельно
    String cleaned = value.replaceAll("[\\s\\-]", ""); //
убираем дефисы
    return ISBN_10.matcher(cleaned).matches()
        || ISBN_13.matcher(cleaned).matches();
}
}
```

Шаг 3. Использовать в DTO:

```
public record BookRequest(
    @NotBlank String title,
    @NotBlank @ValidIsbn String isbn, // ← наш валидатор
    @NotNull Long authorId
) {}
```

2.8 OpenAPI-конфигурация в контракте

BooksApiContractConfig — класс без @Component. Он несёт только аннотации.

```
@OpenAPIDefinition(
    info = @Info(title = "Books API", version = "1.0.0", ...)
    servers = { @Server(url = "http://localhost:8080",
description = "Local") }
)
@SecurityScheme(
    name = BooksApiContractConfig.SECURITY_SCHEME_BEARER,
    type = SecuritySchemeType.HTTP,
    scheme = "bearer",
    bearerFormat = "JWT"
```



```
)  
    public final class BooksApiContractConfig {  
        public static final String SECURITY_SCHEME_BEARER =  
"bearerAuth";  
    }
```

springdoc-openapi сканирует весь classpath (включая подключённые JAR) и автоматически подхватывает @OpenAPIDefinition. Сервис-реализатор получает готовый Swagger UI, не написав ни строчки про документацию.

В методах API константа используется так:

```
@Operation(security = @SecurityRequirement(name =  
BooksApiContractConfig.SECURITY_SCHEME_BEARER))
```

Задания для самостоятельной работы

Вы должны реализовать контракты на основе данного примера уже для своего основного сервиса. В завершении у вас должен быть проект, который описывает все DTO и интерфейсы контроллеров как требование к вашему основному API сервису. Подключите данный проект к реализации и реализуйте все интерфейсы.



Реализация REST-контракта с HATEOAS на Spring

Предыдущее занятие было посвящено созданию библиотеки контракта (*books-api-contract*). Здесь мы реализуем этот контракт в сервисе *demo-rest* и разберём полную архитектуру - от входящего HTTP-запроса до сформированного HATEOAS-ответа.

1. Введение

REST API (ресурсы + HTTP-глаголы) - стандарт, который знает и использует большинство разработчиков. Но тут есть проблема - все клиенты жёстко зашивают URL («/api/books», «/api/authors/1»). Стоит переименовать ресурс - все клиенты ломаются. Такое часто случается в командах, которые в основном состоят из бекенд разработчиков, которые не думают о клиентском коде.

HATEOAS решает это - **ответ сервера сам содержит ссылки** на следующие действия. Клиент знает наизусть только одну точку входа (GET /api). Остальные URL он получает из поля `_links` ответа.

Это не академическая концепция, а реальная практика, её используют крупные публичные API - GitHub REST API возвращает `_links` с `html_url`, `commits_url`, `pulls_url`. Amazon API Gateway сам строит HAL-ответы. А API Spotify и вовсе следует почти всем принципам HATEOAS, поэтому и рекомендуется к изучению.

Настройка среды

Мы вам дадим готовый проект вокруг работы с авторами и книгами. Делаем по шагам:

Шаг 1. Открыть проект в IDEA

1. **File → Open → выберите папку demo-rest (корень репозитория, где лежит корневой pom.xml).**
2. **IDEA спросит «Trust this project?» - подтвердите.**
3. **В панели Maven (справа) вы увидите два модуля: books-api-contract и demo-rest.**

Если открыли не корень, а один из подмодулей - все модули не будут видны.

Закройте и откройте правильную папку.

Шаг 2. Собрать все модули одной командой

Команда для Windows где используется обертка, в Linux/Mac - `mvn`
`.\mvnw install`



Maven сначала соберёт books-api-contract → установит его в репозиторий локальный ~/.m2 → затем скомпилирует demo-rest, который подключает проект контрактов как зависимость из локального репозитория.

Шаг 3. Запустить сервис (в Linux/Mac - mvn)

```
.\mvnw spring-boot:run -pl demo-rest -am
```

- **-pl demo-rest** - собери и запусти только этот модуль
- **-am (--also-make)** - сначала собери все его зависимости (books-api-contract)

Можно и так:

```
.\mvnw spring-boot:run
```

Сервис стартовал (если порт 8080 не занят) откройте <http://localhost:8080/api> и <http://localhost:8080/swagger-ui.html>.

2. Теория

2.1 Архитектура проекта

Проект разделён на два Maven-модуля – контракт и реализацию контракта. Реализация - demo-rest зависит от books-api-contract как от обычной библиотеки (<dependency> в pom.xml). Контракт не знает ничего о реализации - это односторонняя зависимость.

2.2 Корневой агрегатор POM и Maven Reactor

Зачем нужен корневой pom.xml?

Без него приходится собирать модули вручную в правильном порядке:

Без агрегатора просто неудобно:

```
cd books-api-contract && .\mvnw install
```

```
cd ../demo-rest && .\mvnw spring-boot:run
```

А с агрегатором одна команда из корня:

```
.\mvnw spring-boot:run -pl demo-rest -am
```

Устройство корневого pom.xml:

```
<groupId>edu.rutmiit.demo</groupId>
```

```
<artifactId>demo-rest-root</artifactId>
```

```
<version>1.0</version>
```



```
<packaging>pom</packaging>
<modules>
  <module>books-api-contract</module>
  <module>demo-rest</module>
</modules>
```

<packaging>pom</packaging> - агрегатор, не приложение!

Порядок важен в modules, зависимости указываются раньше.

Maven Reactor - часть Maven, которая:

1. Читает корневой pom.xml и находит все <module>
2. Для каждого модуля читает его pom.xml и выявляет зависимости
3. Строит граф: books-api-contract → demo-rest
4. Собирает модули в топологическом порядке (зависимости первыми)

Не путаем агрегатор и родительский POM. Каждый demo-rest/pom.xml и books-api-contract/pom.xml по-прежнему наследует spring-boot-starter-parent. Корневой POM только объединяет их в один Maven-запуск - он не является их Java-родителем.

Зависимость demo-rest от контракта

В demo-rest/pom.xml:

```
<dependency>
  <groupId>edu.rutmiit.demo</groupId>
  <artifactId>books-api-contract</artifactId>
  <version>1.0</version>
</dependency>
```

Когда Reactor собирает demo-rest, он уже установил books-api-contract в локальный кэш Maven (~/.m2). Поэтому demo-rest находит артефакт контракта как обычную библиотеку.

2.3 Слой хранения

В демонстрационных целях вместо базы данных используется InMemoryStorage - Spring-компонент, хранящий данные в ConcurrentHashMap (коллекции в памяти, без БД):

@Component



```
public class InMemoryStorage {  
    public final Map<Long, AuthorResponse> authors = new  
ConcurrentHashMap<>();  
    public final Map<Long, BookResponse> books = new  
ConcurrentHashMap<>();  
  
    public final AtomicLong authorSequence = new AtomicLong(0);  
    public final AtomicLong bookSequence = new AtomicLong(0);  
  
    @PostConstruct  
    public void init() {  
        // заполняем тестовыми данными при старте  
    }  
}
```

Ключевые детали:

- **ConcurrentHashMap** - потокобезопасная хэш-мапа. Потокобезопасность важна, потому что HTTP-сервер обрабатывает запросы в нескольких потоках одновременно. Обычный HashMap здесь вызвал бы ошибки гонки данных.
- **AtomicLong** - атомарный счётчик для генерации ID. `incrementAndGet()` - атомарная операция - прибавить 1 и вернуть результат. Безопасна в многопоточной среде (в отличие от `long id++`).
- **@PostConstruct** - аннотация Spring. Метод с ней вызывается после того, как Spring создал бин и выполнил инъекцию зависимостей, но до того, как бин стал доступен другим компонентам. Идеальное место для инициализации.

Данные живут только пока работает приложение. При перезапуске всё сбрасывается до начального состояния `@PostConstruct`.

Трюк с booksCount - поле `booksCount` в `AuthorResponse` хранится статично - оно устанавливается при инициализации (`booksCount(2)`) и **не пересчитывается** при добавлении или удалении книг. Это намеренное упрощение для демонстрации. В реальном приложении либо хранится в БД с автоматическим пересчётом, либо вычисляется на лету запросом `COUNT`.



2.4 Уровни зрелости REST и HATEOAS

Леонард Ричардсон описал четыре уровня зрелости REST-API:

Уровень	Название	Описание
0	«Болото»	Один URL, все запросы - POST с разными командами в теле
1	Ресурсы	Разные URL: /books, /authors, /books/1
2	HTTP-глаголы	GET, POST, PUT, PATCH, DELETE + правильные коды ответов
3	HATEOAS	В ответе есть _links с URL следующих возможных действий

Наш проект реализует **уровень 3**.

HATEOAS (Hypermedia As The Engine Of Application State) - в ответ включаются гиперссылки:

// Уровень 2 - просто данные:

```
{ "id": 1, "firstName": "Лев", "lastName": "Толстой" }
```

// Уровень 3 - данные + навигация:

```
{
  "id": 1,
  "firstName": "Лев",
  "lastName": "Толстой",
  "_links": {
    "self": { "href": "http://localhost:8080/api/authors/1"
  },
  "books": { "href":
"http://localhost:8080/api/books?authorId=1&page=0&size=20" },
  "collection": { "href":
"http://localhost:8080/api/authors?page=0&size=20" }
  }
}
```

Клиент «переходит» от ресурса к ресурсу по этим ссылкам, не зная ни одного URL наизусть, в этом и есть трюк «навигации».



2.5 DTO-классы Response

В контракте есть два вида DTO:

Тип	Класс	Почему
Request (входящие данные)	AuthorRequest, BookRequest, PatchAuthorRequest...	record - неизменяемые, компактные
Response (исходящие данные)	AuthorResponse, BookResponse	Обычный класс с Lombok - потому что нужно наследование

Java record не может расширять другой класс. Но для HATEOAS нам нужно:

```
public class AuthorResponse extends  
RepresentationModel<AuthorResponse> { ... }
```

RepresentationModel<T> - базовый класс Spring HATEOAS, добавляющий хранилище ссылок _links. Без него поле _links не появится в JSON.

Поэтому Response-классы выглядят так:

```
@Getter  
@Builder  
@EqualsAndHashCode(callSuper = false)  
@JsonInclude(JsonInclude.Include.NON_NULL)  
@Relation(collectionRelation = "authors", itemRelation =  
"author")  
@Schema(description = "Информация об авторе")  
public class AuthorResponse extends  
RepresentationModel<AuthorResponse> {  
    private final Long id;  
    private final String firstName;  
    private final String lastName;  
    private final String fullName;  
    private final String email;  
    private final String bio;  
    private final LocalDate birthDate;  
    private final String nationality;
```



```
private final Integer booksCount;  
}
```

Рассмотрим аннотации:

- **@Getter (Lombok)** - генерирует `getId()`, `getFirstName()` и т.д. Нужны потому что поля `private final` - геттер единственный способ их прочитать извне.
- **@Builder (Lombok)** – генерирует строителя (паттерн для создания сложных объектов):
 - `AuthorResponse.builder().id(1L).firstName("Лев").build()`. Используется повсюду в сервисах.
- **@EqualsAndHashCode(callSuper = false)** - сравнение двух `AuthorResponse` не должно учитывать `_links`. Без этой аннотации (или с `callSuper = true`) два объекта с одинаковыми данными, но разными ссылками оказались бы «разными» - что будет неожиданно.
- **@JsonInclude(NON_NULL)** - Jackson не включает в JSON поля со значением `null`. Если у автора нет `bio` - поле просто не появится в ответе, вместо `"bio": null`.
- **@Relation(collectionRelation = "authors", itemRelation = "author")** - задаёт имена в HAL-ответах. Без неё Spring HATEOAS назвал бы список `authorResponseList`. С ней - `authors` в `_embedded`.

2.6 Валидация

Request-DTO - Java recordy с аннотациями Bean Validation:

```
public record AuthorRequest(  
  
    @NotBlank(message = "Имя автора не может быть пустым")  
    @Size(max = 100, message = "Имя не может превышать 100  
символов")  
    String firstName,  
  
    @NotBlank(message = "Фамилия автора не может быть пустой")  
    @Size(max = 100, message = "Фамилия не может превышать 100  
символов")  
    String lastName,
```



```
@Email(message = "Некорректный формат email")
@Size(max = 255, message = "Email не может превышать 255
символов")
String email,

@Size(max = 2000, message = "Биография не может превышать
2000 символов")
String bio,

@Past(message = "Дата рождения должна быть в прошлом")
LocalDate birthDate,

@Size(max = 100)
String nationality
) {}
```

Как работает валидация:

1. Клиент отправляет POST /api/authors с JSON-телом.
2. Spring десериализует JSON → AuthorRequest.
3. Аннотация @Valid в контракте (@RequestBody @Valid AuthorRequest request) запускает Bean Validation.
4. Если нарушение - Spring бросает MethodArgumentNotValidException.
5. GlobalExceptionHandler перехватывает его и возвращает 400 Bad Request с ErrorResponse.

PATCH-запросы используют отдельный record PatchAuthorRequest, где **все поля необязательны** (нет @NotBlank, есть только @Size и т.д.):

```
public record PatchAuthorRequest(
    @Size(max = 100) String firstName,    // null = не менять
    @Size(max = 100) String lastName,    // null = не менять
    @Email String email,                  // null = не менять
    // ...
) {}
```



Если клиент передаёт только { "bio": "Новая биография" }, поля firstName, lastName, email будут null в Java - и сервис оставит их без изменений.

2.7 Кастомная валидация

Стандартных аннотаций Bean Validation (@NotBlank, @Pattern...) иногда недостаточно. Для ISBN создан собственный валидатор:

Аннотация:

```
@Documented
@Constraint(validatedBy = IsbnValidator.class)
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidIsbn {
    String message() default "Некорректный ISBN. Допустимые
форматы: ISBN-10 или ISBN-13";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

Три метода (message, groups, payload) - обязательные для любой constraint-аннотации Bean Validation.

Реализатор:

```
public class IsbnValidator implements
ConstraintValidator<ValidIsbn, String> {

    private static final Pattern ISBN_10 =
Pattern.compile("^\\d{9}[\\dX]$");
    private static final Pattern ISBN_13 =
Pattern.compile("^97[89]\\d{10}$");

    @Override
    public boolean isValid(String value,
ConstraintValidatorContext context) {
        if (value == null || value.isBlank()) {
```



```
        return true;    // null/пусто - ответственность
@NotBlank, не наша
    }
    String cleaned = value.replaceAll("[\\s\\-]", "");    //
"978-5-389-06259-8" → "9785389062598"
    return ISBN_10.matcher(cleaned).matches()
        || ISBN_13.matcher(cleaned).matches();
    }
}
```

Использование в BookRequest:

```
@NotBlank(message = "ISBN не может быть пустым")
```

```
@ValidIsbn
```

```
String isbn,
```

Обе аннотации применяются независимо. @NotBlank проверяет первым, @ValidIsbn - что строка является валидным ISBN.

2.8 Обработка исключений. @ControllerAdvice и RFC 7807

Когда что-то идёт не так (автор не найден, ISBN уже существует, ошибка валидации) - нужно вернуть клиенту структурированное сообщение об ошибке. Для этого используем @ControllerAdvice. Но что возвращать? Это подскажет стандарт!

ErrorResponse - формат RFC 7807 Problem Details

Определён в контракте (чтобы все реализующие сервисы возвращали одинаковый формат):

```
public record ErrorResponse(
    int status,           // HTTP-код: 404, 400, 409, 500
    String type,          // URI-идентификатор типа:
"https://api.example.com/problems/resource-not-found"
    String title,         // Краткое название: "Ресурс не найден"
    String detail,        // Детали: "Author with id=99 not found"
    String instance,      // URL запроса: "/api/authors/99"
    Instant timestamp,    // Момент ошибки (UTC)
    List<FieldError> fieldErrors // null для всех ошибок, кроме
400 с ошибками полей
```



```
) {  
    public record FieldError(String field, Object rejectedValue,  
String message) {}  
}  
@JsonInclude(NON_NULL) - поле fieldErrors не попадёт в JSON если оно null (что  
происходит для 404, 409, 500).
```

Исключения в контракте

Два доменных исключения определены **в контракте**, а не в реализации - потому
что бросаются в сервисном слое, а клиенты (другие сервисы) должны знать их типы:

```
// ResourceNotFoundException: "Author with id=99 not found"  
public class ResourceNotFoundException extends RuntimeException {  
    public ResourceNotFoundException(String resourceName, Object  
resourceId) {  
        super(String.format("%s with id=%s not found",  
resourceName, resourceId));  
    }  
}
```

```
// ISBNAlreadyExistsException: "Book with ISBN=978... already  
exists"  
public class ISBNAlreadyExistsException extends RuntimeException  
{  
    public ISBNAlreadyExistsException(String isbn) {  
        super("Book with ISBN=" + isbn + " already exists");  
    }  
}
```

GlobalExceptionHandler - перехват всех исключений

@ControllerAdvice

```
public class GlobalExceptionHandler {  
  
    private static final String BASE_PROBLEM_URI =  
"https://api.example.com/problems/";
```



```
// 404 - ресурс не найден
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleResourceNotFound(
    ResourceNotFoundException ex, HttpServletRequest req)
{
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(new ErrorResponse(
            404,
            BASE_PROBLEM_URI + "resource-not-found",
            "Ресурс не найден",
            ex.getMessage(),      // "Author with
id=99 not found"
            req.getRequestURI(), //
"/api/authors/99"
            Instant.now(),
            null                  // нет ошибок по
полям
        ));
}

// 409 - ISBN уже существует
@ExceptionHandler(IsbnAlreadyExistsException.class)
public ResponseEntity<ErrorResponse> handleIsbnConflict(
    IsbnAlreadyExistsException ex, HttpServletRequest
req) {
    return ResponseEntity.status(HttpStatus.CONFLICT)
        .body(new ErrorResponse(409,
            BASE_PROBLEM_URI + "isbn-conflict",
            "Конфликт ISBN",
            ex.getMessage(),
            req.getRequestURI(),
```



```
Instant.now(), null));  
  
}  
  
// 400 - ошибка валидации @Valid  
@ExceptionHandler(MethodArgumentNotValidException.class)  
public ResponseEntity<ErrorResponse> handleValidation(  
    MethodArgumentNotValidException ex,  
    HttpServletRequest req) {  
    // Собираем все ошибки полей  
    List<ErrorResponse.FieldError> fieldErrors =  
ex.getBindingResult().getFieldErrors().stream()  
        .map(fe -> new ErrorResponse.FieldError(  
            fe.getField(),          // "isbn"  
            fe.getRejectedValue(),  // "123"  
            fe.getDefaultMessage() // "Некорректный  
ISBN..."  
        ))  
        .toList();  
  
    return ResponseEntity.status(HttpStatus.BAD_REQUEST)  
        .body(new ErrorResponse(400,  
            BASE_PROBLEM_URI + "validation-error",  
            "Ошибка валидации",  
            "isbn: Некорректный ISBN; firstName: Имя  
не может быть пустым",  
            req.getRequestURI(),  
            Instant.now(),  
            fieldErrors // здесь - заполнено  
        ));  
}  
  
// 500 - всё остальное (заглушка для непредвиденных ошибок)
```




```
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleAll(Exception ex,
HttpServletRequest req) {
    // Здесь должно быть логирование: log.error("Unexpected
error", ex);
    return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(new ErrorResponse(500,
                    BASE_PROBLEM_URI + "internal-error",
                    "Внутренняя ошибка сервера",
                    "Произошла непредвиденная ошибка.
Обратитесь к поддержке.",
                    req.getRequestURI(),
                    Instant.now(), null));
}
}
```

Как работает @ControllerAdvice

Spring AOP перехватывает любое исключение, выброшенное из контроллера (включая вложенные вызовы сервисов). Метод с @ExceptionHandler подбирается по типу исключения - от наиболее конкретного к Exception.class.

Порядок перехвата

Spring сначала ищет @ExceptionHandler с точным типом (ResourceNotFoundException), затем с родительским (RuntimeException), затем с Exception. Поэтому handleAll ловит только то, что не поймали специализированные обработчики.

2.9 Слой сервисов, циклическая зависимость и @Lazy

AuthService и BookService зависят друг от друга:

- **AuthService.delete(id)** вызывает **bookService.deleteBooksByAuthorId(id)** - каскадное удаление книг
- **BookService.createBook(request)** вызывает **authorService.findById(request.authorId())** - проверка, что автор существует

Это **циклическая зависимость**. Spring не может создать оба бина одновременно:

- **Чтобы создать AuthService - нужен BookService**



- Чтобы создать BookService - нужен AuthorService

Решение - @Lazy:

@Service

```
public class AuthorService {  
    private final InMemoryStorage storage;  
    private final BookService bookService;
```

```
        public AuthorService(InMemoryStorage storage, @Lazy  
BookService bookService) {  
            this.storage = storage;  
            this.bookService = bookService;  
        }  
    }
```

@Service

```
public class BookService {  
    private final InMemoryStorage storage;  
    private final AuthorService authorService;
```

```
        public BookService(InMemoryStorage storage, @Lazy  
AuthorService authorService) {  
            this.storage = storage;  
            this.authorService = authorService;  
        }  
    }
```

@Lazy на одном из параметров заставляет Spring создать CGLIB-прокси вместо реального бина. Когда реально нужен BookService (при первом вызове метода) - Spring создаёт его в этот момент. Цикл разрывается.

@Lazy нужно ставить на менее важный из двух, или на тот, метод которого вызывается реже. Здесь оба сервиса ставят @Lazy - это симметричное решение.



2.10 Пагинация и PagedResponse

В сервисах пагинация реализована вручную (без Spring Data JPA, поскольку хранилище - ConcurrentHashMap):

Собственный PagedResponse<T>

Определён в контракте - понятный, простой record:

```
public record PagedResponse<T>(
    List<T> content,           // элементы текущей страницы
    int     pageNumber,       // номер страницы: 0, 1, 2...
    int     pageSize,         // сколько элементов на странице
    long    totalElements,    // всего элементов в хранилище
    int     totalPages,       // всего страниц
    boolean last              // это последняя страница?
) {}
```

Реализация в AuthorService

```
public PagedResponse<AuthorResponse> findAll(int page, int size)
{
    List<AuthorResponse> all = storage.authors.values().stream()

        .sorted(Comparator.comparingLong(AuthorResponse::getId))
        .toList();

    int totalElements = all.size();
    int totalPages    = size > 0 ? (int) Math.ceil((double)
totalElements / size) : 1;
    int from = page * size;                                // первый
индекс текущей страницы
    int to   = Math.min(from + size, totalElements); // последний
(не выйти за границу)
    List<AuthorResponse> content = (from >= totalElements)
        ? List.of()                                     // запрашиваемая страница за
пределами данных
        : all.subList(from, to); // срез списка
}
```



```
        return new PagedResponse<>(content, page, size,
totalElements, totalPages, page >= totalPages - 1);
    }
```

Мост PagedResponse → Page<T> → PagedModel

PagedResourcesAssembler (Spring HATEOAS) умеет работать только с Page<T> (интерфейс Spring Data). Наш PagedResponse - не Page. Поэтому в контроллере строим мост через PageImpl:

```
PagedResponse<AuthorResponse> paged = authorService.findAll(page,
size);
```

```
// Оборачиваем в стандартный Page<T>:
Page<AuthorResponse> springPage = new PageImpl<>(
    paged.content(), // содержимое страницы
    PageRequest.of(paged.pageNumber(), paged.pageSize()), //
метаданные
    paged.totalElements() // общее число (для расчёта
next/prev)
);
```

```
// PagedResourcesAssembler строит PagedModel и добавляет ссылки
навигации:
```

```
return pagedAuthorsAssembler.toModel(springPage,
authorModelAssembler);
```

PagedResourcesAssembler автоматически вычисляет, есть ли prev и next, на основе totalElements и текущего PageRequest. Ссылки добавляются только если соответствующие страницы существуют.

2.11 Ключевые классы Spring HATEOAS

RepresentationModel<T>

Базовый класс, добавляющий хранилище ссылок. Любой DTO, который должен содержать _links, должен наследовать этот класс.



EntityModel<T>

Обёртка для **одного объекта** с набором ссылок:

```
EntityModel.of(authorResponse,  
    linkTo(methodOn(AuthorController.class).getAuthorById(1L)).withSelfRel(),  
    linkTo(methodOn(BookController.class).getAllBooks(1L, null,  
null, null, 0, 20)).withRel("books"),  
    linkTo(methodOn(AuthorController.class).getAllAuthors(0,  
20)).withRel("collection")  
);
```

В JSON выглядит как данные объекта + поле `_links`.

PagedModel<EntityModel<T>>

Обёртка для **страницы** объектов. Содержит `_embedded`, `_links` (с `first/prev/next/last`) и `page` с метаданными.

@Relation

```
@Relation(collectionRelation = "authors", itemRelation =  
"author")
```

```
public class AuthorResponse ...
```

- `collectionRelation = "authors"` → ключ внутри `_embedded` при коллекции
- `itemRelation = "author"` → имя при встраивании внутрь другого объекта (например, `BookResponse.author`)

2.12 Паттерн Assembler

Без Assembler код превращения DTO → EntityModel дублируется везде. **С Assembler** - единый центр формирования ссылок:

@Component

```
public class AuthorModelAssembler  
    implements RepresentationModelAssembler<AuthorResponse,  
EntityModel<AuthorResponse>> {
```

```
    @Override
```

```
    public EntityModel<AuthorResponse> toModel(AuthorResponse  
author) {
```



```
return EntityModel.of(author,
    // self: GET /api/authors/{id}
    linkTo(methodOn(AuthorController.class)

    .getAuthorById(author.getId())).withSelfRel(),
    // books: GET /api/books?authorId={id}&page=0&size=20
    linkTo(methodOn(BookController.class)
        .getAllBooks(author.getId(), null, null,
null, 0, 20)).withRel("books"),
    // collection: GET /api/authors?page=0&size=20
    linkTo(methodOn(AuthorController.class)
        .getAllAuthors(0, 20)).withRel("collection")
    );
}
```

`linkTo(methodOn(AuthorController.class).getAuthorById(author.getId()))` - это **не вызов метода**. `methodOn(...)` создаёт CGLIB-прокси контроллера, а Spring HATEOAS анализирует какой метод вызывается, читает его аннотации (`@GetMapping("/{id}")`, `@PathVariable`) и строит URL. Аргументы подставляются в шаблон URL.

Если вы переименуете `@GetMapping("/{id}")` в `@GetMapping("/by-id/{id}")` в контракте - все ссылки в `Assembler` обновятся автоматически без изменения кода `Assembler`.

```
@Component
public class BookModelAssembler
    implements RepresentationModelAssembler<BookResponse>,
EntityModel<BookResponse>> {

    @Override
    public EntityModel<BookResponse> toModel(BookResponse book) {
        return EntityModel.of(book,
            linkTo(methodOn(BookController.class)
                .getBookById(book.getId())).withSelfRel(),
```



```
        linkTo(methodOn(AuthorController.class)

.getAuthorById(book.getAuthor().getId())).withRel("author"),
        linkTo(methodOn(BookController.class)
                .getAllBooks(null, null, null, null, 0,
20)).withRel("collection")
    );
}
```

2.13 Реализация контракта в контроллере

Интерфейс из контракта описывает **что**, контроллер описывает **как**:

```
@RestController // нет @RequestMapping - унаследован из AuthorApi
public class AuthorController implements AuthorApi {
    // Конструкторная инъекция - Spring видит единственный конструктор
и
    // автоматически инжектирует все параметры (без @Autowired в Spring
Boot)

    public AuthorController(AuthorService authorService,
                            BookService bookService,
                            AuthorModelAssembler authorModelAssembler,
                            BookModelAssembler bookModelAssembler,
                            PagedResourcesAssembler<AuthorResponse>
pagedAuthorsAssembler,
                            PagedResourcesAssembler<BookResponse>
pagedBooksAssembler) { ... }

    @Override
    public PagedModel<EntityModel<AuthorResponse>> getAllAuthors(int
page, int size) {
        PagedResponse<AuthorResponse> paged =
authorService.findAll(page, size);
        Page<AuthorResponse> springPage = new PageImpl<>(
            paged.content(), PageRequest.of(paged.pageNumber(),
paged.pageSize()),

```



```
        paged.totalElements());
        return pagedAuthorsAssembler.toModel(springPage,
authorModelAssembler);
    }

    @Override
    public EntityModel<AuthorResponse> getAuthorById(Long id) {
        // Сервис бросает ResourceNotFoundException если не найден -
GlobalExceptionHandler поймает
        return
authorModelAssembler.toModel(authorService.findById(id));
    }

    @Override
    public ResponseEntity<EntityModel<AuthorResponse>>
createAuthor(AuthorRequest request) {
        AuthorResponse created = authorService.create(request);
        EntityModel<AuthorResponse> model =
authorModelAssembler.toModel(created);
        return ResponseEntity
            .created(model.getRequiredLink("self").toUri()) //
Location header
            .body(model);
    }

    @Override
    public EntityModel<AuthorResponse> updateAuthor(Long id,
AuthorRequest request) {
        // PUT: заменяем все поля (authorService.update строит новый
объект из request)
        return authorModelAssembler.toModel(authorService.update(id,
request));
    }

    @Override
```




```
        public EntityModel<AuthorResponse> patchAuthor(Long id,
PatchAuthorRequest request) {
            // PATCH: обновляем только переданные поля (null = оставить как
есть)

            return
authorModelAssembler.toModel(authorService.patchAuthor(id, request));
        }

        @Override
        public void deleteAuthor(Long id) {
            authorService.delete(id); // каскадно удаляет книги через
bookService
        }

        @Override
        public PagedModel<EntityModel<BookResponse>> getBooksByAuthor(Long
id, int page, int size) {
            authorService.findById(id); // явная проверка: если автора нет
→ 404

            PagedResponse<BookResponse> paged =
                bookService.findAllBooks(id, null, null, null, page,
size);

            Page<BookResponse> springPage = new PageImpl<>(
                paged.content(),
                PageRequest.of(paged.pageNumber(), paged.pageSize()),
                paged.totalElements());

            return
                pagedBooksAssembler.toModel(springPage,
bookModelAssembler);
        }
    }
}
```

Почему нет @GetMapping, @PostMapping и т.д. в контроллере? Аннотации наследуются из интерфейса AuthorApi. Spring MVC сканирует аннотации как на классе-реализаторе, так и на интерфейсе. Дублировать их в контроллере – ошибка, так как возникнет конфликт маппингов и приложение не запустится.



2.14 PUT vs PATCH

Разница видна в AuthorService:

PUT - создаём новый объект полностью из данных запроса:

```
public AuthorResponse update(Long id, AuthorRequest request) {  
    findById(id); // проверка существования  
  
    // Все поля берём из request - старые данные теряются:  
    AuthorResponse updated = AuthorResponse.builder()  
        .id(id)  
        .firstName(request.firstName()) // обязательное  
поле  
        .lastName(request.lastName()) // обязательное  
поле  
        .fullName(request.firstName() + " " +  
request.lastName())  
        .email(request.email()) // null если не  
передан → поле исчезнет из JSON  
        .bio(request.bio()) // null если не  
передан → поле исчезнет из JSON  
        .birthDate(request.birthDate())  
        .nationality(request.nationality())  
        // booksCount не передаётся в AuthorRequest → будет  
null  
        .build();  
  
    storage.authors.put(id, updated);  
    return updated;  
}
```

PATCH - читаем существующий объект и заменяем только то, что пришло:

```
public AuthorResponse patchAuthor(Long id, PatchAuthorRequest  
request) {
```



```
AuthorResponse existing = findById(id); // получаем текущие  
данные
```

```
// null-coalescing: если в request поле null → берём из  
existing:
```

```
String newFirstName = request.firstName() != null ?  
request.firstName() : existing.getFirstName();
```

```
String newLastName = request.lastName() != null ?  
request.lastName() : existing.getLastName();
```

```
AuthorResponse updated = AuthorResponse.builder()  
    .id(id)  
    .firstName(newFirstName)  
    .lastName(newLastName)  
    .fullName(newFirstName + " " + newLastName)  
    .email(request.email() != null ?  
request.email() : existing.getEmail())  
    .bio(request.bio() != null ? request.bio()  
: existing.getBio())  
    .birthDate(request.birthDate() != null ?  
request.birthDate() : existing.getBirthDate())  
    .nationality(request.nationality() != null ?  
request.nationality() : existing.getNationality())  
    .booksCount(existing.getBooksCount()) // всегда  
берём из existing  
    .build();
```

```
storage.authors.put(id, updated);  
return updated;
```

```
}
```

2.15 Сервис книг. Проверка ISBN и каскадное удаление

Проверка уникальности ISBN в BookService:



```
private void validateIsbn(String isbn, Long currentBookId) {
    storage.books.values().stream()
        .filter(b -> b.getIsbn().equalsIgnoreCase(isbn))
        // currentBookId != null при обновлении - исключаем
текущую книгу из проверки
        .filter(b -> !b.getId().equals(currentBookId))
        .findAny()
        .ifPresent(b -> { throw new
IsbnAlreadyExistsException(isbn); });
}
```

При createBook - validateIsbn(isbn, null) (проверяем среди всех).
При updateBook - validateIsbn(isbn, id) (исключаем саму себя - обновление той же книги с тем же ISBN - не конфликт).

Каскадное удаление автора:

```
// AuthorService.delete:
public void delete(Long id) {
    findById(id); // 404 если не
существует
    bookService.deleteBooksByAuthorId(id); // сначала удаляем все
книги автора
    storage.authors.remove(id); // затем самого автора
}
```

```
// BookService.deleteBooksByAuthorId:
public void deleteBooksByAuthorId(Long authorId) {
    List<Long> toDelete = storage.books.values().stream()
        .filter(b -> b.getAuthor() != null &&
b.getAuthor().getId().equals(authorId))
        .map(BookResponse::getId)
        .toList();
    toDelete.forEach(storage.books::remove); // удаляем каждую
книгу автора
}
```



| СОП

```
}
```

Заметьте порядок - сначала собираем список ID в `toList()` (материализуем), потом удаляем. Нельзя итерировать `ConcurrentHashMap.values()` и одновременно удалять из него - порядок операций непредсказуем.

3. Практический разбор

Пройдём полный цикл работы с API. Все запросы выполняйте через Swagger UI.

Шаг 1. Точка входа GET /api

<http://localhost:8080/api>, <http://localhost:8080/swagger-ui/index.html>

The screenshot shows the Swagger UI interface for a REST API. The top bar indicates the selected endpoint is `GET /api`. Below this, the 'Parameters' section is empty. The 'Execute' button is highlighted. The 'Responses' section shows a successful response with status code 200. The response body is a JSON object with the following structure:

```
{
  "_links": {
    "authors": {
      "href": "http://localhost:8080/api/authors?page=0&size=20"
    },
    "books": {
      "href": "http://localhost:8080/api/books?page=0&size=20{&authorId,genre,publishedYear,titleSearch}",
      "templated": true
    }
  }
}
```

The response headers are also visible:

```
connection: keep-alive
content-type: application/hal+json
date: Tue, 10 Mar 2020 13:13:39 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

```
{
```

```
  "_links": {
    "authors": {
      "href": "http://localhost:8080/api/authors?page=0&size=20"
    },
    "books": {
```



```
        "href":  
        "http://localhost:8080/api/books?page=0&size=20{&authorId,genre,publis  
hedYear,titleSearch}",  
        "templated": true  
    }  
}  
}
```

RootController возвращает RepresentationModel<?> - это объект **ТОЛЬКО** со ссылками, без данных. Клиент знает только этот URL, все остальные он открывает из _links.

```
@GetMapping  
public RepresentationModel<?> getRoot() {  
    RepresentationModel<?> root = new RepresentationModel<>();  
    root.add(  
        linkTo(methodOn(AuthorController.class).getAllAuthors(0,  
20)).withRel("authors"),  
  
        linkTo(methodOn(BookController.class).getAllBooks(null,null,null,null,  
0, 20)).withRel("books")  
    );  
    return root;  
}
```

Шаг 2. Список авторов GET /api/authors?page=0&size=2

http://localhost:8080/api/authors?page=0&size=2

Ответ:

```
{  
  "_embedded": {  
    "authors": [  
      {  
        "_links": {  
          "self": {  
            "href": "http://localhost:8080/api/authors/1"  
          },  
        },  
      },  
    ],  
  },  
}
```



```
    "books": {
      "href":
"http://localhost:8080/api/books?authorId=1&page=0&size=20{&genre,publ
ishedYear,titleSearch}",
      "templated": true
    },
    "collection": {
      "href":
"http://localhost:8080/api/authors?page=0&size=20"
    }
  },
  "bio": "Русский писатель, один из наиболее известных
авторов в мировой литературе.",
  "birthDate": "1828-09-09",
  "booksCount": 2,
  "firstName": "Лев",
  "fullName": "Лев Толстой",
  "id": 1,
  "lastName": "Толстой",
  "nationality": "Русский"
},
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/authors/2"
    },
    "books": {
      "href":
"http://localhost:8080/api/books?authorId=2&page=0&size=20{&genre,publ
ishedYear,titleSearch}",
      "templated": true
    },
  },
}
```



```
        "collection": {
            "href":
"http://localhost:8080/api/authors?page=0&size=20"
        }
    },
    "bio": "Русский писатель, мыслитель, философ и
публицист.",
    "birthDate": "1821-11-11",
    "booksCount": 1,
    "firstName": "Фёдор",
    "fullName": "Фёдор Достоевский",
    "id": 2,
    "lastName": "Достоевский",
    "nationality": "Русский"
    }
]
},
"_links": {
    "self": {
        "href": "http://localhost:8080/api/authors?page=0&size=2"
    }
},
"page": {
    "number": 0,
    "size": 2,
    "totalElements": 2,
    "totalPages": 1
}
}
```

Обратите внимание - поле email отсутствует - у авторов в начальных данных нет email, и @JsonInclude(NON_NULL) не включает null-поля.



Шаг 3: Создание автора POST /api/authors

POST http://localhost:8080/api/authors

```
{
  "firstName": "Антон",
  "lastName": "Чехов",
  "email": "chekhov@example.com",
  "birthDate": "1860-01-29"
}
```

Ответ (201 Created):

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/authors/3"
    },
    "books": {
      "href":
"http://localhost:8080/api/books?authorId=3&page=0&size=20{&genre,publ
ishedYear,titleSearch}",
      "templated": true
    },
    "collection": {
      "href": "http://localhost:8080/api/authors?page=0&size=20"
    }
  },
  "birthDate": "1860-01-29",
  "booksCount": 0,
  "firstName": "Антон",
  "fullName": "Антон Чехов",
  "id": 3,
  "lastName": "Чехов"
}
```



```
}
```

Что произошло:

1. Spring десериализовал JSON → AuthorRequest record.
2. @Valid запустил Bean Validation - всё в порядке.
3. authorService.create(request) → storage.authorSequence.incrementAndGet() вернул 3 → создал AuthorResponse с id=3 через Lombok @Builder → положил в storage.authors.put(3, author).
4. authorModelAssembler.toModel(created) добавил ссылки.
5. model.getRequiredLink("self").toUri() → URI("http://localhost:8080/api/authors/3") → ResponseEntity.created(uri) → заголовок Location.

Шаг 4. Ошибка валидации

POST http://localhost:8080/api/authors

```
{  
  "firstName": "",  
  "lastName": "Чехов"  
}
```

Ответ (400 Bad Request):

```
{  
  "status": 400,  
  "type": "https://api.example.com/problems/validation-error",  
  "title": "Ошибка валидации",  
  "detail": "firstName: Имя автора не может быть пустым",  
  "instance": "/api/authors",  
  "timestamp": "2026-03-10T13:18:50.539664500Z",  
  "fieldErrors": [  
    {  
      "field": "firstName",  
      "rejectedValue": "",  
      "message": "Имя автора не может быть пустым"  
    }  
  ]  
}
```



```
}
```

Шаг 5. Создание книги с невалидным ISBN

POST http://localhost:8080/api/books

```
{  
  "title": "Вишнёвый сад",  
  "isbn": "123",  
  "authorId": 3  
}
```

Ответ (400 Bad Request):

```
{  
  "status": 400,  
  "type": "https://api.example.com/problems/validation-error",  
  "title": "Ошибка валидации",  
  "detail": "isbn: Некорректный ISBN. Допустимые форматы: ISBN-10  
(10 цифр или 9 цифр + X) или ISBN-13 (начинается с 978/979, 13 цифр)",  
  "instance": "/api/books",  
  "timestamp": "2026-03-10T13:19:37.465623300Z",  
  "fieldErrors": [  
    {  
      "field": "isbn",  
      "rejectedValue": "123",  
      "message": "Некорректный ISBN. Допустимые форматы: ISBN-10  
(10 цифр или 9 цифр + X) или ISBN-13 (начинается с 978/979, 13 цифр)"  
    }  
  ]  
}
```

Шаг 6. Создание книги

POST http://localhost:8080/api/books

```
{
```



| **соп**

```
"title": "Вишнёвый сад",  
"isbn": "9785389012345",  
"authorId": 3,  
"genre": "Пьеса",  
"publishedYear": 1904,  
"language": "ru"  
}
```

Ответ (201 Created):



```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/books/4"
    },
    "author": {
      "href": "http://localhost:8080/api/authors/3"
    },
    "collection": {
      "href":
"http://localhost:8080/api/books?page=0&size=20{&authorId,genre,publis
hedYear,titleSearch}",
      "templated": true
    }
  },
  "author": {
    "birthDate": "1860-01-29",
    "booksCount": 0,
    "firstName": "Антон",
    "fullName": "Антон Чехов",
    "id": 3,
    "lastName": "Чехов"
  },
  "createdAt": "2026-03-10T16:21:08.8420109",
  "genre": "Пьеса",
  "id": 4,
  "isbn": "9785389012345",
  "language": "ru",
  "publishedYear": 1904,
  "title": "Вишнёвый сад"
}
```

Шаг 7. PATCH - частичное обновление



PATCH <http://localhost:8080/api/authors/3>

```
{
  "bio": "Великий русский писатель и драматург"
}
```

PatchAuthorRequest - только bio не-null. В patchAuthor:

- **firstName = null** → берём `existing.getFirstName()` = "Антон"
- **lastName = null** → берём `existing.getLastName()` = "Чехов"
- **bio = "Великий..."** → заменяем

Ответ 200 OK с обновлённым объектом. Email (у него не появился), дата рождения - не тронуты.

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/authors/3"
    },
    "books": {
      "href":
"http://localhost:8080/api/books?authorId=3&page=0&size=20{&genre,publ
ishedYear,titleSearch}",
      "templated": true
    },
    "collection": {
      "href": "http://localhost:8080/api/authors?page=0&size=20"
    }
  },
  "bio": "Великий русский писатель и драматург",
  "birthDate": "1860-01-29",
  "booksCount": 0,
  "firstName": "Антон",
```



```
"fullName": "Антон Чехов",  
"id": 3,  
"lastName": "Чехов"  
}
```

Шаг 8. Удаление с каскадом

DELETE http://localhost:8080/api/authors/3

В сервисе AuthorService:

1. **findByld(3)** - проверяем существование
2. **bookService.deleteBooksByAuthorId(3)** - собирает все книги автора 3, удаляет книгу с id=4
3. **storage.authors.remove(3)** - удаляет автора

Ответ 204 No Content. Тело пустое.

Шаг 9. 404 при запросе удалённого ресурса

GET http://localhost:8080/api/authors/3

```
{  
  "status": 404,  
  "type": "https://api.example.com/problems/resource-not-found",  
  "title": "Ресурс не найден",  
  "detail": "Author with id=3 not found",  
  "instance": "/api/authors/3",  
  "timestamp": "2026-03-10T13:26:09.112506400Z"  
}
```

fieldErrors отсутствует - @JsonInclude(NON_NULL) не включает null в JSON.

4. Справочные материалы

Все ключевые классы и аннотации

Элемент	Где	Назначение
extends RepresentationModel<T>	DTO Response	Добавляет _links в JSON



@Relation(collectionRelation, itemRelation)	DTO Response	Имена ключей в _embedded
@EqualsAndHashCode(callSuper = false)	DTO Response	Исключает _links из equals
@JsonInclude(NON_NULL)	DTO Response, ErrorResponse	Не включать null-поля в JSON
@Getter @Builder (Lombok)	DTO Response	Геттеры + fluent-строитель
EntityModel<T>	Controller, Assembler	Единственный объект + ссылки
PagedModel<EntityModel<T>>	Controller	Страница объектов + ссылки навигации
RepresentationModelAssembler<T, M>	Assembler	Паттерн DTO → Model
linkTo(methodOn(Ctrl.class).method()).withSelfRel()	Assembler	Типобезопасная self-ссылка
linkTo(...).withRel("name")	Assembler	Именованная ссылка
PagedResourcesAssembler<T>	Controller	Строит PagedModel из Page
PageImpl<T>	Controller	Мост PagedResponse → Page
@ControllerAdvice	GlobalExceptionHandler	Перехват исключений



@ExceptionHandler(X.class)	GlobalExceptionHandler	Обработчик конкретного типа
@Lazy	Service constructor	Разрыв циклической зависимости
@PostConstruct	InMemoryStorage	Инициализация после создания бина
ConcurrentHashMap	InMemoryStorage	Потокобезопасное хранилище
AtomicLong	InMemoryStorage	Потокобезопасный счётчик ID
@Constraint(validatedBy = X.class)	ValidIsbn	Связь аннотации с реализатором
ConstraintValidator<A, T>	IsbnValidator	Реализация кастомной валидации
implements AuthorApi / BookApi	Controller	Контракт-ориентированная реализация

5. Задания для самостоятельной работы

Задания выполняются в проекте *demo-rest* (реализация контракта).

Задание 1. Исследование связки контракт и реализация

1. Откройте `AuthorController.java`. Для каждого метода заполните таблицу:

Метод контроллера	Метод интерфейса AuthorApi	HTTP-глагол + URL	Какие зависимости используются (сервисы, ассемблеры)
-------------------	----------------------------	-------------------	--



getAllAuthors	getAllAuthors	GET /api/authors	...
getAuthorById	...	...	...
createAuthor	...	...	...
...			

1. Прделайте то же самое для BookController.java и BookApi.
2. Ответьте: почему в AuthorController нет аннотаций @GetMapping, @PostMapping и т.д.? Что произойдёт, если их добавить?

Задание 2. Расширение HATEOAS-ссылок в Assembler

1. Откройте AuthorModelAssembler.java. Сейчас он возвращает три ссылки: self, books, collection.
2. Добавьте ссылку update - на PUT /api/authors/{id}. Используйте `linkTo(methodOn(AuthorController.class).updateAuthor(author.getId(), null)).withRel("update")`.
3. Добавьте ссылку delete - на DELETE /api/authors/{id}. Используйте аналогичный подход.
4. Запустите приложение, выполните GET /api/authors/1 и убедитесь, что в `_links` появились новые ссылки.
5. Вопрос - HATEOAS-клиент увидит ссылку delete и поймёт, что ресурс можно удалить. Какую проблему это может создать, если удаление доступно не всем пользователям? Как её решить?

Задание 3. Реализация нового метода контракта

Это задание является продолжением Задания 3 из Лабораторной 1 (добавить метод поиска авторов по имени в AuthorApi).

1. Если вы ещё не добавили метод `searchByName` в AuthorApi.java - добавьте (см. Лабораторную 1, задание 3).
2. В AuthorService.java добавьте метод `searchByName(String query, int page, int size)`. Логика: фильтруйте авторов, у которых `fullName` содержит `query` (case-insensitive), затем примените пагинацию.
3. В AuthorController.java реализуйте новый метод из интерфейса. Не забудьте про мост `PagedResponse` → `PagedImpl` → `PagedModel`.



4. Проверьте через Swagger: GET /api/authors/search?name=Лев - должен вернуться только Толстой.

Задание 4. Динамический подсчёт booksCount

Сейчас поле booksCount в AuthorResponse задаётся статически при инициализации и не обновляется при добавлении/удалении книг.

1. В AuthorService.java добавьте метод recalculateBooksCount(Long authorId).
Логика:
 1. Получить текущего автора через findById
 2. Посчитать сколько книг в storage.books принадлежат этому автору
 3. Создать обновлённый AuthorResponse с правильным booksCount и сохранить в storage.authors
2. Вызовите recalculateBooksCount в BookService:
 1. После createBook - пересчитать для request.authorId()
 2. После deleteBook - пересчитать для автора удалённой книги
 3. После deleteBooksByAuthorId - пересчитать (или обнулить) booksCount для автора
3. Проверьте:
 1. Создайте книгу для автора 1, booksCount в ответе GET /api/authors/1 увеличился
 2. Удалите книгу, booksCount уменьшился

Задание 5. Реализация BookSummaryResponse в контроллере

Это задание является продолжением Задания 5 из Лабораторной 1 (создать BookSummaryResponse и метод в BookApi).

1. Если вы ещё не создали BookSummaryResponse и метод getAllBooksSummary() в BookApi - сделайте это (см. Лабораторную 1, задание 5).
2. В пакете assemblers создайте BookSummaryModelAssembler, реализующий RepresentationModelAssembler<BookSummaryResponse, EntityModel<BookSummaryResponse>>. Ссылки: self на полную книгу GET /api/books/{id}, collection, на GET /api/books/summary.



3. В `BookService.java` добавьте метод, который возвращает список `BookSummaryResponse` (маппинг из `BookResponse` - только `id`, `title`, `isbn`).
4. В `BookController.java` реализуйте метод `getAllBooksSummary()`. Верните `List<EntityModel<BookSummaryResponse>>`, используя новый `assembler`.
5. Сравните ответы `GET /api/books` и `GET /api/books/summary`. Выпишите: какие поля исчезли в `summary`?



Разработка GraphQL-контракта и язык запросов

В предыдущих занятиях мы создали REST-контракт (books-api-contract) и реализовали его с HATEOAS в demo-rest. REST прекрасно работает, но у него есть фундаментальные ограничения: клиент всегда получает **фиксированный** набор полей, для связанных данных нужны **дополнительные запросы**, а для разных клиентов приходится создавать **разные эндпоинты** (/books/summary, /books/full).

GraphQL решает все три проблемы одним контрактом - **схемой**, в которой описаны типы данных, связи между ними и доступные операции. Клиент сам определяет, какие поля ему нужны, в одном запросе.

GraphQL используется в Netflix (DGS Framework), GitHub (GitHub API v4), Spotify, Shopify, Airbnb, Meta. По данным State of JavaScript 2024, **более 60% фронтенд-разработчиков** используют GraphQL хотя бы в одном проекте.

Необходимое ПО и настройка среды

Запуск проекта:

```
.\mvnw spring-boot:run -pl demo-rest -am
```

После старта:

- **REST API:** <http://localhost:8080/swagger-ui.html>
- **GraphQL IDE:** <http://localhost:8080/graphiql> - интерактивная среда для выполнения GraphQL-запросов с автодополнением и документацией по схеме

2. Теория

2.1 REST vs GraphQL - две парадигмы, одна задача

Обе технологии решают одну задачу - предоставить клиенту доступ к данным сервера по HTTP. Но подходы радикально различаются:

Характеристика	REST	GraphQL
Точки входа	Множество URL: /books, /authors/{id}	Один URL: /graphql
Формат запроса	HTTP-метод + URL + query-параметры	POST с телом { "query": "..."} }



Выбор полей	Сервер решает, что возвращать	Клиент перечисляет нужные поля
Связанные данные	Дополнительные HTTP-запросы	Вложенные поля в одном запросе
Over-fetching	Есть - лишние поля приходят всегда	Нет - приходит ровно то, что запрошено
Under-fetching	Есть - нужны N+1 запросов для связей	Нет - одним запросом получаем всё
Контракт	OpenAPI/Swagger (YAML/JSON)	SDL-схема (.graphqls)
Кеширование	HTTP-кеширование из коробки	Нужна специальная инфраструктура (Apollo Cache)

Пример over-fetching в REST:

Мобильное приложение показывает список книг. Ему нужны только title и isbn. REST вернёт все поля (description, genre, publishedYear, language, createdAt, updatedAt, полного автора со всеми его полями, HATEOAS-ссылки) - для мобильного трафика это расточительство.

Пример under-fetching в REST:

Страница «Автор и его книги» - нужны данные автора и список его книг. REST потребует минимум два запроса:

GET /api/authors/1

GET /api/authors/1/books

GraphQL решает обе проблемы одним запросом:

```
{
  author(id: "1") {
    firstName
    lastName
    books {
      content {
        title
        isbn
      }
    }
  }
}
```



| соп

```
    }  
  }  
}  
}
```

Ответ содержит **ровно** запрошенные поля, ни больше ни меньше:

```
{  
  "data": {  
    "author": {  
      "firstName": "Лев",  
      "lastName": "Толстой",  
      "books": {  
        "content": [  
          { "title": "Война и мир", "isbn": "978-5-389-06259-8"  
        },  
          { "title": "Анна Каренина", "isbn": "978-5-389-04374-0"  
        }  
      ]  
    }  
  }  
}
```

2.2 Как работает GraphQL - один эндпоинт, один метод

В REST у каждого ресурса свой URL и HTTP-метод. В GraphQL **все запросы** идут на один URL одним методом:

POST /graphql

Content-Type: application/json

```
{  
  "query": "{ authors { content { fullName } } }"  
}
```



Сервер разбирает строку query, находит соответствующие резолверы (аналоги контроллеров), выполняет их и возвращает JSON ровно той формы, которую запросил клиент.

Ключевое отличие от REST: клиент не «ходит» по разным URL - он описывает форму нужных данных, а сервер эту форму заполняет.

2.3 GraphQL Schema Definition Language (SDL)

Схема - это контракт GraphQL API. Она описывает:

- **Какие типы данных существуют**
- **Какие поля есть у каждого типа**
- **Какие операции доступны (Query, Mutation)**
- **Какие входные данные принимают мутации**

Схема хранится в файлах .graphqls (или .graphql) и напоминает TypeScript-интерфейсы.

Скалярные типы

Встроенные «атомарные» типы данных:

Тип	Описание	Пример
Int	Целое число (32-бит знаковое)	42, -1
Float	Число с плавающей точкой (IEEE 754)	3.14, -0.5
String	Строка в UTF-8	"Война и мир"
Boolean	Логическое значение	true, false
ID	Уникальный идентификатор (строка)	"1", "uuid-..."

ID - это семантический String. GraphQL всегда передаёт его как строку, даже если в БД это число. Это важно - в запросах ID **всегда в кавычках**: book(id: "1").

Пользовательские скаляры

Встроенных типов для дат нет. Мы объявляем свои:

scalar DateTime

Для java.time.LocalDateTime - "2026-03-10T16:30:00"

scalar Date

Для java.time.LocalDate - "1828-09-09"



Серверная реализация (DGS, Spring for GraphQL) регистрирует правила сериализации: как преобразовать строку "1828-09-09" в `LocalDate.of(1828, 9, 9)` и обратно.

Объектный тип (type)

Описывает сущность с набором полей:

```
type Author {
```

```
  id: ID! # ! означает NOT NULL - поле
```

обязательно

```
  firstName: String!
```

```
  lastName: String!
```

```
  fullName: String!
```

```
  email: String # без ! - может быть null
```

```
  bio: String
```

```
  birthDate: Date
```

```
  nationality: String
```

```
  booksCount: Int!
```

```
}
```

Правила ! (Non-Null):

- **String!** - сервер гарантирует, что это поле никогда не будет null
- **String** - поле может быть null
- **[Book!]!** - список не-null, элементы не-null (но список может быть пустым [])
- **[Book]** - и список и элементы могут быть null

Связи между типами

GraphQL описывает связи прямо в типе - не нужны отдельные эндпоинты:

```
type Author {
```

```
  id: ID!
```

```
  firstName: String!
```

```
  # ...
```

```
  # Связь «один ко многим»: книги данного автора с пагинацией
```

```
  books(page: Int = 0, size: Int = 20): BookConnection!
```

```
}
```



```
type Book {  
  id: ID!  
  title: String!  
  # ...  
  
  # Связь «многие к одному»: автор книги  
  author: Author!  
}
```

Связь `Author.books` - это **поле с аргументами**. Клиент может запросить книги автора на конкретной странице: `books(page: 1, size: 5)`. Значения `= 0` и `= 20` - это значения по умолчанию: если клиент их не передаст, подставятся автоматически.

Важно: связи определяются на уровне **схемы** (контракта). Реализация (резолверы на сервере) загружает данные только если клиент фактически запросил это поле. В REST все поля включаются в ответ всегда.

Пагинация в GraphQL

```
type BookConnection {  
  content: [Book!]!           # элементы текущей страницы  
  pageInfo: PageInfo!         # метаданные страницы  
  totalElements: Int!          # общее количество  
}
```

```
type PageInfo {  
  pageNumber: Int!  
  pageSize: Int!  
  totalPages: Int!  
  last: Boolean!               # это последняя страница?  
}
```

Это наша собственная модель пагинации, аналогичная `PagedResponse<T>` из REST-контракта. В индустрии также часто используется **Relay-спецификация** (`edges`, `cursor`, `node`), но `offset`-пагинация проще для понимания.



Входные типы (input)

Для входных данных мутаций используется ключевое слово `input`, а не `type`:

```
input CreateBookInput {  
    title: String!           # обязательное поле  
    isbn: String!  
    authorId: ID!  
    description: String      # необязательное поле  
    genre: String  
    publishedYear: Int  
    language: String  
}
```

```
input UpdateBookInput {  
    title: String!  
    isbn: String!  
    description: String  
    genre: String  
    publishedYear: Int  
    language: String  
}
```

Почему `input`, а не `type`? Это разделение «входных» и «выходных» данных. Объектные типы (`type`) могут содержать связи и резолверы. Входные типы (`input`) - только скалярные поля и другие `input`. GraphQL-сервер обрабатывает их по-разному.

Сравнение с REST:

REST DTO	GraphQL input	Назначение
BookRequest	CreateBookInput	Создание книги
UpdateBookRequest	UpdateBookInput	Полное обновление (PUT)
AuthorRequest	CreateAuthorInput	Создание автора
-	BookFilter	Фильтрация списка книг

```
input BookFilter {  
    authorId: ID           # фильтр по автору
```



СОП

```
genre: String          # по жанру
publishedYear: Int      # по году
titleSearch: String     # поиск в названии
```

```
}
```

В REST фильтры передаются query-параметрами

(?genre=Роман&publishedYear=1869). В GraphQL - как аргумент с input-типом.

Корневые типы: Query и Mutation

Query - точка входа для **чтения** данных (аналог GET в REST):

```
type Query {
  # Получить книгу по ID - аналог GET /api/books/{id}
  book(id: ID!): Book

  # Список книг с фильтрацией и пагинацией - аналог GET
  # /api/books?...
  books(filter: BookFilter, page: Int = 0, size: Int = 20):
  BookConnection!

  # Получить автора по ID - аналог GET /api/authors/{id}
  author(id: ID!): Author

  # Список авторов - аналог GET /api/authors
  authors(page: Int = 0, size: Int = 20): AuthorConnection!
}
```

Mutation - точка входа для **изменения** данных (аналог POST/PUT/DELETE):

```
type Mutation {
  # POST /api/books
  createBook(input: CreateBookInput!): Book!

  # PUT /api/books/{id}
  updateBook(id: ID!, input: UpdateBookInput!): Book!

  # DELETE /api/books/{id}
}
```



СОП

```
deleteBook(id: ID!): Boolean!
```

```
# POST /api/authors
```

```
createAuthor(input: CreateAuthorInput!): Author!
```

```
# PUT /api/authors/{id}
```

```
updateAuthor(id: ID!, input: UpdateAuthorInput!): Author!
```

```
# DELETE /api/authors/{id}
```

```
deleteAuthor(id: ID!): Boolean!
```

```
}
```

Ключевое отличие от REST: мутация createBook возвращает полный объект Book! - клиент сразу запрашивает нужные поля результата. В REST контроллер сам решает, что включать в ответ 201 Created, а клиент не может на это влиять.

Комментарии

```
# Однострочный комментарий - не попадает в документацию
```

```
"""
```

Многострочный комментарий (description).

Попадает в интроспекцию и отображается в GraphQL.

```
"""
```

```
type Author {
```

```
  "Идентификатор автора - строковое описание для одной строки"
```

```
  id: ID!
```

```
}
```

Для документации предпочтительнее """описание""" - такие комментарии видны клиентам через интроспекцию и отображаются в GraphQL IDE.

2.4 Язык запросов GraphQL

Мы определили схему - это контракт на стороне сервера. Теперь разберём, как клиент формулирует запросы.

Простой запрос (query)

Ключевое слово query можно опустить для анонимного запроса:



Полная форма

```
query GetAllAuthors {  
  authors {  
    content {  
      id  
      fullName  
    }  
  }  
}
```

Сокращённая форма (анонимный запрос)

```
{  
  authors {  
    content {  
      id  
      fullName  
    }  
  }  
}
```

Обе формы идентичны. Имя операции (GetAllAuthors) - произвольное, но рекомендуется для отладки и логирования.

Аргументы

Аргументы передаются в скобках после имени поля:

```
{  
  book(id: "1") {  
    title  
    isbn  
    genre  
  }  
}
```



```
books(filter: { genre: "Роман", publishedYear: 1869 }, page: 0,
size: 5) {
  content {
    title
    publishedYear
  }
  totalElements
}
```

Вложенные поля (traversal)

Главная сила GraphQL - навигация по связям в одном запросе:

```
{
  author(id: "1") {
    fullName
    nationality
    books(page: 0, size: 10) {
      content {
        title
        isbn
        genre
      }
      totalElements
      pageInfo {
        totalPages
        last
      }
    }
  }
}
```

Один HTTP-запрос заменяет два REST-вызова: GET /api/authors/1 + GET /api/authors/1/books.



Мутации (mutation)

Мутации синтаксически похожи на запросы, но явно помечены ключевым словом mutation:

```
mutation {  
  createAuthor(input: {  
    firstName: "Антон"  
    lastName: "Чехов"  
    email: "chekhov@example.com"  
    birthDate: "1860-01-29"  
    nationality: "Русский"  
  }) {  
    id  
    fullName  
    booksCount  
  }  
}
```

Ответ:

```
{  
  "data": {  
    "createAuthor": {  
      "id": "3",  
      "fullName": "Антон Чехов",  
      "booksCount": 0  
    }  
  }  
}
```

Клиент сам решил, какие поля нового автора ему нужны - только id, fullName и booksCount.

Переменные (variables)

Хардкодить значения в строку запроса - плохая практика. Переменные отделяют **структуру** запроса от **данных**:

```
mutation CreateBook($input: CreateBookInput!) {
```




```
createBook(input: $input) {  
  id  
  title  
  author {  
    fullName  
  }  
}
```

Переменные передаются отдельным JSON-полем variables:

```
{  
  "query": "mutation CreateBook($input: CreateBookInput!) {  
createBook(input: $input) { id title } }",  
  "variables": {  
    "input": {  
      "title": "Вишнёвый сад",  
      "isbn": "978-5-389-01234-5",  
      "authorId": "3",  
      "genre": "Пьеса",  
      "publishedYear": 1904,  
      "language": "ru"  
    }  
  }  
}
```

Формат объявления переменных: \$имя: Тип! - знак \$ обязателен, тип должен совпадать с ожидаемым аргументом в схеме.

В GraphQL переменные вводятся в нижней панели «Variables».

Алиасы (aliases)

Если нужно запросить одно и то же поле с разными аргументами - используйте алиасы:

```
{  
  tolstoy: author(id: "1") {  
    fullName
```



```
        booksCount
    }
    dostoevsky: author(id: "2") {
        fullName
        booksCount
    }
}
```

Ответ:

```
{
  "data": {
    "tolstoy": {
      "fullName": "Лев Толстой",
      "booksCount": 2
    },
    "dostoevsky": {
      "fullName": "Фёдор Достоевский",
      "booksCount": 1
    }
  }
}
```

Без алиасов два вызова `author(id: ...)` конфликтовали бы по имени ключа в JSON.

Фрагменты (fragments)

Фрагменты устраняют дублирование - задаёте набор полей один раз и используете

в нескольких местах:

```
fragment AuthorShort on Author {
  id
  fullName
  nationality
  booksCount
}

{
```



```
tolstoy: author(id: "1") {  
  ...AuthorShort  
}  
dostoevsky: author(id: "2") {  
  ...AuthorShort  
}  
}
```

Оператор ... (spread) разворачивает фрагмент - подставляет его поля вместо себя.

Фрагмент привязан к конкретному типу (on Author).

2.5 Структура HTTP-ответа GraphQL

Любой ответ GraphQL имеет фиксированную структуру:

```
{  
  "data": { ... },           // результат запроса (может быть null  
при ошибке)  
  "errors": [ ... ],         // список ошибок (опционально)  
  "extensions": { ... }     // метаданные сервера (опционально)  
}
```

Успешный ответ:

```
{  
  "data": {  
    "author": {  
      "fullName": "Лев Толстой"  
    }  
  }  
}
```

Ответ с ошибкой (автор не найден):

```
{  
  "data": {  
    "author": null  
  },  
  "errors": [  
    {
```



```
"message": "Author with id=99 not found",
"extensions": {
  "classification": "NOT_FOUND"
}
}
]
}
```

Отличие от REST: в REST ошибка - это HTTP-код 404 и тело ErrorResponse. В GraphQL HTTP-код **всегда 200**, а ошибки передаются в поле errors. Это одна из самых обсуждаемых особенностей GraphQL.

2.6 Интроспекция

GraphQL-сервер умеет описывать **сам себя**. Специальные запросы, начинающиеся с `__`, позволяют узнать полную схему:

```
{
  __schema {
    queryType { name }
    mutationType { name }
    types {
      name
      kind
    }
  }
}

{
  __type(name: "Author") {
    name
    fields {
      name
      type {
        name
        kind
      }
    }
  }
}
```



В продакшене интроспекцию часто отключают (`spring.graphql.schema.introspection.enabled=false`), чтобы злоумышленник не мог изучить полную структуру API.

2.7 Безопасность GraphQL

GraphQL гибок, но эта гибкость создаёт уникальные риски, которых нет в REST:

Атака глубокой вложенностью

Если в схеме есть циклические связи (Author $\rightarrow$ books $\rightarrow$ author $\rightarrow$ books $\rightarrow$...), злоумышленник может отправить запрос с огромной вложенностью:

Атака: рекурсия глубиной 24 уровня

[illegible]





Атака сложностью

Даже неглубокий запрос может быть «дорогим» - например, запросить 10 000 книг с полными данными авторов. Сервер может ограничить **стоимость** запроса (каждому полю и аргументу назначается вес, их сумма не должна превышать лимит).

В REST этих проблем нет - сервер контролирует, что возвращать. В GraphQL клиент контролирует форму ответа, поэтому сервер должен защищаться дополнительно.

3. Практический разбор

Все запросы выполняйте в GraphiQL: <http://localhost:8080/graphiql>

Шаг 1. Знакомство с GraphiQL

Откройте <http://localhost:8080/graphiql> в браузере. Вы увидите три панели:

- Слева - редактор запросов (с автодополнением по Ctrl+Space)
- Справа - результат выполнения
- Снизу - панель переменных (Variables) и заголовков (Headers)
- Боковая панель (Docs) - документация по схеме, сгенерированная автоматически

Нажмите на кнопку «Docs» слева - откроется документация. Найдите Query, кликните - увидите все доступные запросы. Это результат **интроспекции** - GraphiQL прочитал схему с сервера.

Шаг 2. Первый запрос - список авторов

Введите в левой панели:

```
{
  authors {
    content {
      id
      fullName
      nationality
      booksCount
    }
  }
  totalElements
```



| **соп**

```
}  
}
```

Нажмите кнопку ► (Execute). Ожидаемый результат:

```
{  
  "data": {  
    "authors": {  
      "content": [  
        {  
          "id": "1",  
          "fullName": "Лев Толстой",  
          "nationality": "Русский",  
          "booksCount": 2  
        },  
        {  
          "id": "2",  
          "fullName": "Фёдор Достоевский",  
          "nationality": "Русский",  
          "booksCount": 1  
        }  
      ],  
      "totalElements": 2  
    }  
  }  
}
```

Обратите внимание: в ответе **нет** полей email, bio, birthDate - мы их не запрашивали. В REST эти поля пришли бы все.

Шаг 3. Запрос автора с его книгами - навигация по связям

```
{  
  author(id: "1") {  
    firstName  
    lastName  
    fullName  
  }  
}
```




```
books(page: 0, size: 10) {  
  content {  
    title  
    isbn  
    genre  
    publishedYear  
  }  
  totalElements  
  pageInfo {  
    totalPages  
    last  
  }  
}  
}
```

Один запрос - автор и все его книги. В REST потребовались бы два отдельных HTTP-запроса.

Шаг 4. Создание автора - мутация

```
mutation {  
  createAuthor(input: {  
    firstName: "Антон"  
    lastName: "Чехов"  
    nationality: "Русский"  
    birthDate: "1860-01-29"  
  }) {  
    id  
    fullName  
    booksCount  
  }  
}
```

Результат:

```
{
```



```
"data": {  
  "createAuthor": {  
    "id": "3",  
    "fullName": "Антон Чехов",  
    "booksCount": 0  
  }  
}
```

Шаг 5. Создание книги для нового автора

```
mutation {  
  createBook(input: {  
    title: "Вишнёвый сад"  
    isbn: "978-5-389-01234-5"  
    authorId: "3"  
    genre: "Пьеса"  
    publishedYear: 1904  
    language: "ru"  
  }) {  
    id  
    title  
    isbn  
    createdAt  
    author {  
      fullName  
    }  
  }  
}
```

Мы запросили `author { fullName }` внутри результата мутации - сервер вернёт автора вместе с книгой. В REST-ответе автор тоже встроен, но клиент не может выбрать, какие поля автора ему нужны.

Шаг 6. Алиасы - два автора в одном запросе

```
{
```



```
tolstoy: author(id: "1") {  
  fullName  
  booksCount  
}  
dostoevsky: author(id: "2") {  
  fullName  
  booksCount  
}  
}
```

Шаг 7. Переменные - чистая параметризация

В панели запроса:

```
query GetAuthor($authorId: ID!) {  
  author(id: $authorId) {  
    fullName  
    nationality  
    books {  
      content {  
        title  
      }  
      totalElements  
    }  
  }  
}
```

В панели Variables (внизу):

```
{  
  "authorId": "1"  
}
```

Попробуйте менять "authorId" на "2", "3" - запрос не меняется, меняются только данные.

Шаг 8. Проверка защиты от глубокой вложенности

```
{  
  authors {
```



| **СОН**

```
content {  
  books {  
    content {  
      author {  
        books {  
          content {  
            author {  
              books {  
                content {  
                  author {  
                    books {  
                      content { title }  
                    }  
                  }  
                }  
              }  
            }  
          }  
        }  
      }  
    }  
  }  
}
```



```
    }  
  }  
}  
}
```

Ожидаемый результат - ошибка:

```
{  
  "errors": [{  
    "message": "maximum query depth exceeded 24 > 20"  
  }]  
}
```

Сервер **отклонил** запрос ещё до выполнения. Без этой защиты злоумышленник мог бы вызвать экспоненциальный рост нагрузки на сервер.

Шаг 9. Удаление с каскадом

```
mutation {  
  deleteAuthor(id: "3")  
}
```

Результат: { "data": { "deleteAuthor": true } }. Автор Чехов и его книга «Вишнёвый сад» удалены. Проверьте:

```
{  
  author(id: "3") {  
    fullName  
  }  
}
```

Получите ошибку NOT_FOUND - автор удалён.

4. Задания для самостоятельной работы

Задания выполняются в проекте books-api-contract (файл src/main/resources/graphql/schema.graphqls) и в GraphQL.

Задание 1. Исследование существующей схемы

1. Откройте `books-api-contract/src/main/resources/graphql/schema.graphqls`.
2. Для каждого поля типа Query заполните таблицу:



Поле схемы	Аргументы	Тип возврата	Аналог в REST API
book	id: ID!	Book	GET /api/books/{id}
books	...	...	...
author	...	...	...
authors	...	...	...

1. Сравните `CreateBookInput` и `UpdateBookInput`. Какое поле есть только в `CreateBookInput`? Почему его нет в `UpdateBookInput`?
2. В GraphQL выполните интроспекцию-запрос, чтобы программно получить все поля типа `Author`:

```
{
  __type(name: "Author") {
    fields {
      name
      type { name kind }
    }
  }
}
```

Задание 2. Составление запросов

Напишите GraphQL-запросы для каждого сценария. Выполните их в GraphQL и запишите ответ:

1. Получите список всех книг, включая только поля `title`, `isbn` и `fullName` автора.
2. Получите автора с `id: "1"` вместе со всеми полями и его книгами (только `title` и `genre`).
3. Используя алиасы, получите данные автора 1 и автора 2 в одном запросе (только `fullName` и `booksCount`).
4. Используя переменные, напишите универсальный запрос для получения книги по ID. Протестируйте с `id: "1"` и `id: "2"`.

Задание 3. Мутации

1. Создайте нового автора с именем, фамилией потока (мастера / бакалавры) и национальностью. В ответе запросите `id`, `fullName`.



2. Создайте книгу для этого автора. В ответе запросите `id`, `title`, `isbn` и `fullName` автора.
3. Обновите созданную книгу - измените `title`. Убедитесь, что `isbn` остался прежним.
4. Удалите автора. Проверьте, что его книга тоже удалена (запросите книгу по `ID` - должна быть ошибка).

Задание 4. Расширение схемы - новый input-тип `AuthorFilter`

В текущей схеме у запроса `authors` нет фильтрации (в отличие от `books`).

1. Создайте входной тип `AuthorFilter` с полями:
 1. `nationality: String` - фильтрация по национальности
 2. `nameSearch: String` - поиск по имени (`fullName`)
2. Добавьте аргумент `filter: AuthorFilter` к полю `authors` в типе `Query`.
3. Запишите, как будет выглядеть запрос авторов с фильтрацией по национальности "Русский".

Задание 5. Проектирование нового типа `Review`

Спроектируйте расширение схемы - систему рецензий на книги.

1. Создайте тип `Review` с полями: `id: ID!`, `text: String!`, `rating: Int!` (от 1 до 5), `createdAt: DateTime!`.
2. Добавьте связь `reviews: [Review!]!` в тип `Book`.
3. Создайте input `CreateReviewInput` с полями: `bookId: ID!`, `text: String!`, `rating: Int!`.
4. Добавьте мутацию `createReview(input: CreateReviewInput!): Review!` в тип `Mutation`.
5. Добавьте запрос `reviews(bookId: ID!): [Review!]!` в тип `Query`.
6. Напишите (на бумаге/в файле) пример GraphQL-запроса, который получает книгу с `id: "1"`, её `title`, и все рецензии с `text` и `rating`.

5. Дополнительные материалы

Шпаргалка по SDL (Schema Definition Language)

Конструкция	Описание
<code>scalar DateTime</code>	Пользовательский скалярный тип



type Author { ... }	Объектный тип с полями
input CreateBookInput { ... }	Входной тип для аргументов
type Query { ... }	Корневые запросы (чтение)
type Mutation { ... }	Корневые мутации (изменение)
field: String!	Обязательное поле (не может быть null)
field: String	Опциональное поле (может быть null)
field: [Book!]!	Обязательный список обязательных элементов
field(arg: Int = 10): Type	Поле с аргументом и значением по умолчанию
# комментарий	Однострочный комментарий
""описание""	Описание для документации

Шпаргалка по языку запросов

Конструкция	Описание
{ field { subfield } }	Анонимный запрос (сокращённая форма)
query Name { ... }	Именованный запрос
mutation Name { ... }	Именованная мутация
field(arg: "value")	Передача аргументов
query(\$var: Type!) { field(arg: \$var) }	Переменные
alias: field(arg: "value")	Алиас для одноимённых полей
fragment Name on Type { ... }	Определение фрагмента
...FragmentName	Использование фрагмента (spread)
{ __schema { ... } }	Интроспекция схемы
{ __type(name: "Author") { ... } }	Интроспекция конкретного типа

Важные замечания

- ID всегда в кавычках. Даже если в базе хранится число, GraphQL тип ID сериализуется как строка: book(id: "1"), а не book(id: 1).



соп

- GraphQL HTTP-код всегда 200. Ошибки передаются в поле errors, а не через HTTP-статус. Это отличие от REST, где 404/400/500 определяют тип ошибки.
- Интроспекция в продакшене. Отключайте интроспекцию на боевых серверах через `spring.graphql.schema.introspection.enabled=false`.
- Нет PATCH в GraphQL. В REST есть PUT (полная замена) и PATCH (частичное обновление). В GraphQL мутации всегда принимают то, что передано - аналога PATCH-семантики (`null` = «не трогай поле» vs `null` = «установи null») стандарт не определяет. Это решается на уровне дизайна input-типов.



Реализация GraphQL-контракта с Netflix DGS Framework

В предыдущем занятии мы спроектировали GraphQL-схему (контракт) и научились составлять запросы. Теперь мы реализуем этот контракт в сервисе demo-rest, используя **Netflix DGS Framework** - промышленный GraphQL-фреймворк, созданный Netflix для своих микросервисов.

Ключевой архитектурный принцип: GraphQL-слой **не дублирует** бизнес-логику. DataFetcher'ы тонкие - они лишь мапнят GraphQL-запросы на вызовы **существующих** сервисов (BookService, AuthorService), которые мы написали для REST.

1. Введение

Netflix DGS (Domain Graph Service) - это production-ready фреймворк, под которым работают сотни GraphQL-сервисов внутри Netflix. Он интегрирован со Spring Boot, совместим с Spring for GraphQL и предоставляет:

- **Автопривязку схемы к Java-коду**
- **Обработку ошибок и метрик**
- **DataLoader'ы для решения проблемы N+1**
- **Codegen (автогенерация Java-типов из .graphqls)**

DGS - один из двух основных способов реализации GraphQL в экосистеме Spring (второй - Spring for GraphQL). Оба работают поверх GraphQL Java.

Необходимое ПО и настройка среды

Запуск:

```
.\mvnw spring-boot:run -pl demo-rest -am
```

- **REST API: <http://localhost:8080/swagger-ui.html>**
- **GraphQL API: <http://localhost:8080/graphql>**
- **GraphiQL IDE: <http://localhost:8080/graphiql>**

2. Теория

2.1 Архитектура DGS - как схема связывается с Java-кодом

В REST мы писали контроллеры (@RestController), которые реализуют интерфейсы контракта (implements AuthorApi). В DGS вместо контроллеров - **DataFetcher'ы**:

REST	DGS (GraphQL)
------	---------------



@RestController	@DgsComponent
@GetMapping("/books/{id}")	@DgsQuery (метод book)
@PostMapping("/books")	@DgsMutation (метод createBook)
URL + HTTP-метод	Имя поля в GraphQL-схеме
Один контроллер на ресурс	Один DataFetcher на группу полей

DGS при старте приложения:

1. Загружает все .graphqls файлы с classpath
2. Сканирует все @DgsComponent-классы
3. Связывает каждый @DgsQuery/@DgsMutation/@DgsData с соответствующим полем в схеме по имени метода
4. Если имя метода не совпадает - выбрасывает ошибку конфигурации при старте

2.2 Зависимости в pom.xml

Для подключения DGS нужны три элемента:

1. **BOM (Bill of Materials)** - управление версиями всех DGS-зависимостей:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.netflix.graphql.dgs</groupId>
      <artifactId>graphql-dgs-platform-
dependencies</artifactId>
      <version>11.1.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

<scope>import</scope> означает, что это не зависимость, а файл с описанием версий. Все дочерние DGS-зависимости берут версию из этого BOM - не нужно указывать <version> в каждой.



2. Стартер DGS:

```
<dependency>
  <groupId>com.netflix.graphql.dgs</groupId>
  <artifactId>graphql-dgs-spring-graphql-starter</artifactId>
</dependency>
```

Подключает DGS поверх Spring for GraphQL. Включает: авто-обнаружение схемы, DataFetcher-инфраструктуру, серверный GraphQL-endpoint.

3. Расширенные скаляры:

```
<dependency>
  <groupId>com.netflix.graphql.dgs</groupId>
  <artifactId>graphql-dgs-extended-scalars</artifactId>
</dependency>
```

Готовые реализации DateTime, Date, JSON, Long, BigDecimal и других скаляров, которых нет в стандартном GraphQL.

2.3 Конфигурация application.properties

```
# GraphQL - интерактивная IDE для тестирования GraphQL-запросов
spring.graphql.graphiql.enabled=true
```

```
# Путь к GraphQL-endpoint
spring.graphql.path=/graphql
```

```
# Расположение схемы для Spring for GraphQL
spring.graphql.schema.locations=classpath*:graphql/
```

```
# Расположение схемы для DGS
```

```
# По умолчанию DGS ищет в classpath*:schema/**/*.graphql*
```

```
# Переопределяем, чтобы он искал в той же папке, что и Spring for GraphQL
```

```
dgs.graphql.schema-locations=classpath*:graphql/**/*.graphqls
```

```
# Интроспекция - включена для разработки (в продакшене отключать!)
```



```
spring.graphql.schema.introspection.enabled=true
```

Важный момент - схема лежит в books-api-contract/src/main/resources/graphql/schema.graphqls. При сборке она попадает в JAR контракта. demo-rest зависит от контракта как от библиотеки - JAR оказывается в classpath, и DGS находит схему через classpath*: (звёздочка позволяет искать в JAR-зависимостях).

2.4 Основные аннотации DGS

@DgsComponent

Маркирует класс как компонент DGS. По сути - @Component + сигнал для DGS-сканера. Класс с этой аннотацией становится источником DataFetcher'ов.

```
@DgsComponent    // DGS будет сканировать методы этого класса
public class BookDataFetcher {
    // ...
}
```

@DgsQuery

Привязывает метод к полю в type Query. Имя метода должно точно совпадать с именем поля в схеме:

```
// Схема: type Query { book(id: ID!): Book }
@dgsQuery
public BookResponse book(@InputArgument String id) {
    return bookService.findBookById(Long.parseLong(id));
}
```

Если имя метода отличается от имени поля, можно указать явно:

```
@DgsQuery(field = "book")
public BookResponse getBookById(@InputArgument String id) { ... }

@dgsMutation
```

Аналог @DgsQuery, но для type Mutation:

```
// Схема: type Mutation { createBook(input: CreateBookInput!):
Book! }
@dgsMutation
public BookResponse createBook(@InputArgument CreateBookInputGql
input) {
    // ...
}
```



```
}
```

```
@DgsData(parentType, field)
```

Привязывает метод к **вложенному полю** произвольного типа. Используется для резолверов связей:

```
// Схема: type Book { author: Author! }
```

```
@DgsData(parentType = "Book", field = "author")
```

```
public AuthorResponse author(DgsDataFetchingEnvironment dfe) {  
    BookResponse book = dfe.getSource(); // родительский объект  
    return book.getAuthor();  
}
```

```
}
```

```
@InputArgument
```

Маппит GraphQL-аргумент на Java-параметр. DGS автоматически десериализует JSON в нужный Java-тип:

```
@DgsQuery
```

```
public BookConnectionGql books(  
    @InputArgument BookFilterGql filter, // input  
    BookFilter { ... }  
    @InputArgument Integer page, // page: Int = 0  
    @InputArgument Integer size) { ... } // size: Int = 20
```

Для скалярных типов (String, Integer) DGS выполняет прямое преобразование. Для сложных типов (input) - десериализует через Jackson.

```
DgsDataFetchingEnvironment
```

Контекст выполнения DataFetcher'a. Основное применение - получить **родительский объект** во вложенном резолвере:

```
@DgsData(parentType = "Author", field = "books")
```

```
public BookConnectionGql books(DgsDataFetchingEnvironment dfe,  
... ) {  
    AuthorResponse author = dfe.getSource(); // ← родительский  
    Author  
    // загружаем книги этого автора  
}
```



2.5 DataFetcher'ы - корневые и вложенные

Корневой DataFetcher (Query/Mutation)

Обрабатывает **корневые поля** - точки входа в API:

@DgsComponent

```
public class BookDataFetcher {

    private final BookService bookService;

    public BookDataFetcher(BookService bookService) {
        this.bookService = bookService;
    }

    @DgsQuery
    public BookResponse book(@InputArgument String id) {
        return bookService.findBookById(Long.parseLong(id));
    }

    @DgsQuery
    public BookConnectionGql books(
        @InputArgument BookFilterGql filter,
        @InputArgument Integer page,
        @InputArgument Integer size) {

        int pageNum = page != null ? page : 0;
        int pageSize = size != null ? size : 20;

        // Извлекаем параметры фильтрации из input-типа
        Long authorId = null;
        String genre = null;
        Integer publishedYear = null;
        String titleSearch = null;
```



```
        if (filter != null) {
            authorId = filter.authorId() != null
                ? Long.parseLong(filter.authorId()) : null;
            genre = filter.genre();
            publishedYear = filter.publishedYear();
            titleSearch = filter.titleSearch();
        }

        // Переиспользуем существующий сервис - GraphQL не
        дублирует бизнес-логику
        PagedResponse<BookResponse> paged =
bookService.findAllBooks(
            authorId, genre, publishedYear, titleSearch,
            pageNum, pageSize);

        return new BookConnectionGql(
            paged.content(),
            new PageInfoGql(paged.pageNumber(),
            paged.pageSize(),
                paged.totalPages(), paged.last()),
            (int) paged.totalElements());
    }

    @DgsMutation
    public BookResponse createBook(@InputArgument
CreateBookInputGql input) {
        // Маппим GraphQL input → REST DTO
        BookRequest request = new BookRequest(
            input.title(), input.isbn(),
            Long.parseLong(input.authorId()),
            input.description(), input.genre(),
            input.publishedYear(), input.language())
```




```
        );  
        return bookService.createBook(request);  
    }  
  
    @DgsMutation  
    public boolean deleteBook(@InputArgument String id) {  
        bookService.deleteBook(Long.parseLong(id));  
        return true;  
    }  
}
```

Обратите внимание на паттерн - DataFetcher → маппинг типов → вызов сервиса. GraphQL-слой **не знает** как хранятся данные - он лишь транслирует GraphQL-типы в DTO контракта и вызывает сервис.

Вложенный DataFetcher (резолверы связей)

Связи Book.author и Author.books определены в **схеме**, но данные для них нужно загружать **отдельно**. Вложенный резолвер вызывается **только если клиент запросил это поле**:

```
@DgsComponent  
public class BookAuthorDataFetcher {  
  
    private final AuthorService authorService;  
  
    public BookAuthorDataFetcher(AuthorService authorService) {  
        this.authorService = authorService;  
    }  
  
    @DgsData(parentType = "Book", field = "author")  
    public AuthorResponse author(DgsDataFetchingEnvironment dfe)  
{  
        BookResponse book = dfe.getSource();    // родительский  
        объект
```



```
        // В нашем in-memory хранилище автор уже вложен в
BookResponse
        if (book.getAuthor() != null) {
            return book.getAuthor();
        }

        // В реальном приложении здесь был бы вызов к БД или
сервису
        return null;
    }
}

@DgsComponent
public class AuthorBooksDataFetcher {

    private final BookService bookService;

    public AuthorBooksDataFetcher(BookService bookService) {
        this.bookService = bookService;
    }

    @DgsData(parentType = "Author", field = "books")
    public BookConnectionGql books(
        DgsDataFetchingEnvironment dfe,
        @InputArgument Integer page,
        @InputArgument Integer size) {

        AuthorResponse author = dfe.getSource(); // родительский
объект

        int pageNum = page != null ? page : 0;
        int pageSize = size != null ? size : 20;
    }
}
```



```
// Фильтруем книги по ID автора - переиспользуем сервис
PagedResponse<BookResponse> paged =
bookService.findAllBooks(
    author.getId(), null, null, null, pageNum,
    pageSize);

return new BookConnectionGql(
    paged.content(),
    new PageInfoGql(paged.pageNumber(),
    paged.pageSize(),
    paged.totalPages(), paged.last()),
    (int) paged.totalElements());
}
```

2.6 GraphQL-типы как Java records

Для каждого GraphQL-типа, у которого нет прямого аналога в контракте, создаём Java record - неизменяемый контейнер данных:

```
// Соответствует type BookConnection в схеме
public record BookConnectionGql(
    List<BookResponse> content,
    PageInfoGql pageInfo,
    int totalElements
) {}
```

```
// Соответствует type PageInfo в схеме
public record PageInfoGql(
    int pageNumber,
    int pageSize,
    int totalPages,
    boolean last
) {}
```



```
// Соответствует input BookFilter в схеме
public record BookFilterGql(
    String authorId,
    String genre,
    Integer publishedYear,
    String titleSearch
) {}

// Соответствует input CreateBookInput в схеме
public record CreateBookInputGql(
    String title,
    String isbn,
    String authorId,
    String description,
    String genre,
    Integer publishedYear,
    String language
) {}
```

Суффикс Gql отличает GraphQL-типы от REST DTO (которые живут в контракте).

BookResponse - это REST DTO из контракта, BookConnectionGql - обёртка для GraphQL.

Почему record?

Типы GraphQL - неизменяемые контейнеры данных. Java record для этого идеально подходит: компактный синтаксис, автоматические equals/hashCode/toString, иммутабельность. DGS автоматически сериализует поля record'a.

2.7 Регистрация кастомных скаляров

В схеме объявлены scalar DateTime и scalar Date. Сервер должен знать, как преобразовывать строки "2026-03-10T16:30:00" ↔ LocalDateTime и "1828-09-09" ↔ LocalDate:

```
@DgsComponent
public class DateTimeScalarRegistration {
    @DgsRuntimeWiring
```



```
public RuntimeWiring.Builder addScalars(RuntimeWiring.Builder
builder) {
    return builder
        .scalar(ExtendedScalars.DateTime)    // ISO-8601 →
OffsetDateTime
        .scalar(ExtendedScalars.Date);        // ISO-8601 →
LocalDate
    }
}
```

Без этой регистрации DGS не будет знать, как обрабатывать поля createdAt: DateTime и birthDate: Date, и приложение упадёт при старте с ошибкой No scalar implementation found.

2.8 Обработка ошибок в GraphQL

В REST ошибки мапятся на HTTP-коды (@ControllerAdvice → @ExceptionHandler → ResponseEntity<ErrorResponse>). В GraphQL другая модель:

- **HTTP-код всегда 200**
- **Ошибки помещаются в массив "errors" в теле ответа**
- **Каждая ошибка имеет message и extensions.classification**

DGS предоставляет интерфейс DataFetcherExceptionHandler:

```
@Component
public class GraphQLExceptionHandler implements
DataFetcherExceptionHandler {

    @Override
    public CompletableFuture<DataFetcherExceptionHandlerResult>
handleException(
        DataFetcherExceptionHandlerParameters
handlerParameters) {

        Throwable exception = handlerParameters.getException();
```



```
// Ресурс не найден - аналог HTTP 404
if (exception instanceof ResourceNotFoundException) {
    var error = TypedGraphQLError.newNotFoundBuilder()
        .message(exception.getMessage())
        .path(handlerParameters.getPath())    // путь
в запросе: ["author"]
        .build();

    return CompletableFuture.completedFuture(
        DataFetcherExceptionHandlerResult.newResult()
            .error(error)
            .build());
}

// Конфликт ISBN - аналог HTTP 409
if (exception instanceof IsbnAlreadyExistsException) {
    var error = TypedGraphQLError.newConflictBuilder()
        .message(exception.getMessage())
        .path(handlerParameters.getPath())
        .build();

    return CompletableFuture.completedFuture(
        DataFetcherExceptionHandlerResult.newResult()
            .error(error)
            .build());
}

// Всё остальное - внутренняя ошибка (не раскрываем
детали)
var error = TypedGraphQLError.newInternalErrorBuilder()
    .message("Внутренняя ошибка сервера")
    .path(handlerParameters.getPath())
```



```
        .build();

        return CompletableFuture.completedFuture(
            DataFetcherExceptionHandlerResult.newResult()
                .error(error)
                .build());
    }
}
```

Сравнение с REST:

REST (@ControllerAdvice)	GraphQL (DataFetcherExceptionHandler)
ResponseEntity<ErrorResponse> + код 404	TypedGraphQLError.newNotFoundBuilder()
ResponseEntity<ErrorResponse> + код 409	TypedGraphQLError.newConflictBuilder()
ResponseEntity<ErrorResponse> + код 500	TypedGraphQLError.newInternalErrorBuilder()
HTTP-код в заголовке	classification в extensions

Path в ошибке показывает, **в каком месте запроса** произошла ошибка (например, ["author"] или ["createBook", "author"]). Клиент может точно понять, какая часть запроса провалилась.

TypedGraphQLError - утилита DGS. Она автоматически устанавливает правильный classification:

- **newNotFoundBuilder() → classification: "NOT_FOUND"**
- **newConflictBuilder() → classification: "CONFLICT"**
- **newInternalErrorBuilder() → classification: "INTERNAL_ERROR"**

2.9 Защита API - Instrumentation

Instrumentation - механизм graphql-java для перехвата запроса на этапе между парсингом и выполнением. Два готовых инструмента:

```
@Configuration
```

```
public class GraphQLSecurityConfig {
```

```
    /**
```

```
     * Лимит глубины вложенности.
```



СОП

```
*  
* Introspection-запрос GraphQL имеет глубину ~15 уровней,  
* поэтому лимит не может быть ниже 15.
```

```
*/
```

```
@Bean
```

```
public Instrumentation maxQueryDepthInstrumentation() {  
    return new MaxQueryDepthInstrumentation(20);  
}
```

```
/**
```

```
* Лимит сложности (суммарного числа полей).  
* Каждое запрошенное поле добавляет 1 к сложности.
```

```
*/
```

```
@Bean
```

```
public Instrumentation maxQueryComplexityInstrumentation() {  
    return new MaxQueryComplexityInstrumentation(200);  
}
```

```
}
```

Оба бина объявлены как @Bean в @Configuration-классе. Spring Boot DGS автоматически подхватывает все бины типа Instrumentation и регистрирует их в GraphQL-пайплайне.

Как работает защита:

1. Клиент отправляет запрос на /graphql
2. GraphQL-Java парсит запрос → дерево AST
3. MaxQueryDepthInstrumentation обходит дерево и считает максимальную глубину вложенности
4. Если глубина > 20 → возвращает ошибку без выполнения запроса
5. MaxQueryComplexityInstrumentation считает суммарное число полей
6. Если сложность > 200 → возвращает ошибку
7. Если оба лимита не превышены → запрос выполняется



3. Практический разбор

Запустите приложение и откройте GraphQL: <http://localhost:8080/graphql>

Шаг 1. Путь запроса через DGS

Давайте проследим полный путь простого запроса:

```
{
  book(id: "1") {
    title
    isbn
    author {
      fullName
    }
  }
}
```

Что происходит внутри:

1. POST-запрос приходит на `/graphql`
2. DGS парсит строку запроса → AST (дерево)
3. `MaxQueryDepthInstrumentation` проверяет глубину (здесь ~3 - ОК)
4. DGS ищет резолвер для `Query.book` → находит `BookDataFetcher.book()`
5. `BookDataFetcher.book("1")` вызывает `bookService.findBookById(1L)` → `BookResponse`
6. DGS видит, что клиент запросил вложенное поле `author` → ищет резолвер
7. Находит `BookAuthorDataFetcher.author()` → вызывает его с `dfe.getSource()` = `BookResponse`
8. Резолвер возвращает `AuthorResponse` → DGS берёт из него только `fullName`
9. DGS собирает JSON-ответ, включая только запрошенные поля

Если клиент НЕ запросит `author`:

```
{
  book(id: "1") {
    title
    isbn
  }
}
```



```
}  
}
```

Шаги 6–8 **не выполняются вообще** - BookAuthorDataFetcher.author() не вызывается.

Это ключевое отличие от REST, где автор включается в ответ всегда.

Шаг 2. Проверим работу DataFetcher'ов

Корневой Query - список авторов:

```
{  
  authors(page: 0, size: 10) {  
    content {  
      id  
      fullName  
      nationality  
      booksCount  
    }  
    totalElements  
    pageInfo {  
      pageNumber  
      totalPages  
      last  
    }  
  }  
}
```

Обработка: AuthorDataFetcher.authors() → authService.findAll(0, 10) → маппинг в AuthorConnectionGql.

Шаг 3. Вложенный резолвер - автор с книгами

```
{  
  author(id: "1") {  
    fullName  
    nationality  
    books(page: 0, size: 5) {  
      content {  
        title  

```



```
        isbn
        genre
    }
    totalElements
}
}
```

Цепочка вызовов:

1. **AuthorDataFetcher.author("1")** → возвращает **AuthorResponse**
2. **DGS** видит запрос **books(page: 0, size: 5)** → вызывает **AuthorBooksDataFetcher.books()**
3. **dfe.getSource()** = **AuthorResponse** (автор с **id=1**)
4. **bookService.findAllBooks(1L, null, null, null, 0, 5)** → книги Толстого
5. **DGS** берёт только поля **title, isbn, genre** из каждой книги

Шаг 4. Мутация - создание автора

```
mutation {
  createAuthor(input: {
    firstName: "Антон"
    lastName: "Чехов"
    nationality: "Русский"
  }) {
    id
    fullName
    booksCount
  }
}
```

Обработка: **AuthorDataFetcher.createAuthor()** → маппит **CreateAuthorInputGql** в **AuthorRequest** → **authorService.create(request)**.

Шаг 5. Обработка ошибок

Запросим несуществующего автора:

```
{
```



```
author(id: "999") {  
  fullName  
}
```

Ответ:

```
{  
  "data": {  
    "author": null  
  },  
  "errors": [  
    {  
      "message": "Author with id=999 not found",  
      "path": ["author"],  
      "extensions": {  
        "classification": "NOT_FOUND"  
      }  
    }  
  ]  
}
```

Цепочка:

1. **authorService.findById(999) → бросает ResourceNotFoundException**
2. **GraphQLExceptionHandler.handleException()** ловит его
3. **TypedGraphQLError.newNotFoundBuilder() → ошибка с classification: "NOT_FOUND"**

Шаг 6. Дублирование ISBN

```
mutation {  
  createBook(input: {  
    title: "Дубликат"  
    isbn: "978-5-389-06259-8"  
    authorId: "1"  
  }) {  
    id
```



| СОП

```
}  
}
```

ISBN 978-5-389-06259-8 уже занят «Войной и Миром». Ответ:

```
{  
  "errors": [  
    {  
      "message": "Book with ISBN=978-5-389-06259-8 already  
exists",  
      "extensions": {  
        "classification": "CONFLICT"  
      }  
    }  
  ]  
}
```

Шаг 7. Проверка защиты

Глубокий запрос (глубина >20):

```
{  
  authors {  
    content {  
      books {  
        content {  
          author {  
            books {  
              content {  
                author {  
                  books {  
                    content {  
                      author {  
                        books {
```



Ответ: "maximum query depth exceeded 24 > 20" - запрос отклонён до выполнения.

Задания выполняются в проекте *demo-rest* (реализация GraphQL API).

1. Откройте `BookDataFetcher.java` и `AuthorDataFetcher.java`. Для каждого метода заполните таблицу:

2025. Технологии разработки ПО.



book	@DgsQuery	Query.book	bookService.findBookById()
books	@DgsQuery	Query.books	bookService.findAllBooks()
booksByGenre	@DgsQuery	Query.booksByGenre	bookService.findAllBooks()
createBook	@DgsMutation	Mutation.createBook	bookService.createBook()
updateBook	@DgsMutation	Mutation.updateBook	bookService.updateBook()
deleteBook	@DgsMutation	Mutation.deleteBook	bookService.deleteBook()
author	@DgsQuery	Query.author	authorService.findById()
authors	@DgsQuery	Query.authors	authorService.findAll()
createAuthor	@DgsMutation	Mutation.createAuthor	authorService.create()
updateAuthor	@DgsMutation	Mutation.updateAuthor	authorService.update()
deleteAuthor	@DgsMutation	Mutation.deleteAuthor	authorService.delete()

1. Откройте BookAuthorDataFetcher.java. Объясните:

1. Что делает `dfe.getSource()`?
2. Почему тип возвращаемого значения `getSource()` - `BookResponse`, а не `AuthorResponse`?
3. В каком случае этот метод не будет вызван вообще?

2. Сравните GlobalExceptionHandler.java (REST) и GraphQLExceptionHandler.java (GraphQL). В чём ключевые отличия подхода к обработке ошибок?

Задание 2. Добавить DataFetcher для поиска книг по жанру

1. В schema.graphqls добавьте новое поле в type Query:

```
booksByGenre(genre: String!, page: Int = 0, size: Int = 20):  
BookConnection!
```

1. В BookDataFetcher.java добавьте метод booksByGenre с аннотацией @DgsQuery. Он должен:

1. Принимать `genre`, `page`, `size` через `@InputArgument`
2. Вызывать `bookService.findAllBooks(null, genre, null, null, pageNum, pageSize)`



3. Возвращать BookConnectionGql

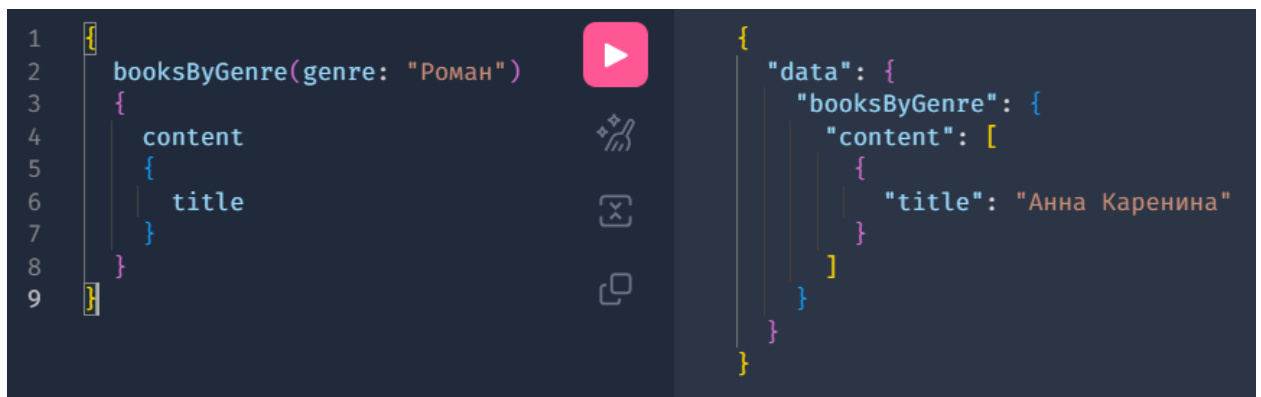
```
@DgsQuery
public BookConnectionGql booksByGenre(
    @InputArgument String genre,
    @InputArgument Integer page,
    @InputArgument Integer size) {

    int pageNum = page != null ? page : 0;
    int pageSize = size != null ? size : 20;

    PagedResponse<BookResponse> paged = bookService.findAllBooks(
        null, genre, null, null, pageNum, pageSize);

    return new BookConnectionGql(
        paged.content(),
        new PageInfoGql(paged.pageNumber(), paged.pageSize(),
            paged.totalPages(), paged.last()),
        (int) paged.totalElements());
}
```

2. Протестируйте в GraphQL: { booksByGenre(genre: "Роман") { content { title } } }



Задание 3. Добавить поле averageRating к типу Author

Реализуйте вычисляемое поле, которого **нет** в AuthorResponse, но оно появится в GraphQL.

1. В `schema.graphqls` добавьте к типу `Author` поле `averageRating: Float`
2. Создайте новый `DataFetcher` `AuthorRatingDataFetcher.java` с `@DgsData(parentType = "Author", field = "averageRating")`
3. В методе верните захардкоженное значение 4.5 (для демонстрации)

```
@DgsComponent
public class AuthorRatingDataFetcher {

    @DgsData(parentType = "Author", field = "averageRating")
    public double averageRating() {
        return 4.5f;
    }
}
```




```
}  
}
```

4. Протестируйте: { author(id: "1") { fullName averageRating } }

```
1 {  
2   author(id: "1")  
3   {  
4     fullName  
5     averageRating  
6   }  
7 }
```

```
{  
  "data": {  
    "author": {  
      "fullName": "Лев Толстой",  
      "averageRating": 4.5  
    }  
  },  
}
```

5. Вопрос: Что произойдёт, если клиент НЕ запросит поле averageRating? Будет ли вызван ваш DataFetcher?

Задание 4. Обработка нового исключения

1. В контракте (books-api-contract) создайте исключение `AuthorHasBooksException(Long authorId)` - бросается, когда пытаются удалить автора, у которого есть книги (без каскадного удаления).

```
public class AuthorHasBooksException extends RuntimeException {  
    public AuthorHasBooksException(Long authorId) {  
        super("You can't delete " + authorId + ". He has books");  
    }  
}
```

2. В `GraphQLExceptionHandler.java` добавьте обработчик для `AuthorHasBooksException`, используя `TypedGraphQLError.newPermissionDeniedBuilder()`.

```
if (exception instanceof AuthorHasBooksException) {  
    var error = TypedGraphQLError.newPermissionDeniedBuilder()  
        .message(exception.getMessage())  
        .path(handlerParameters.getPath())  
        .build();  
  
    return CompletableFuture.completedFuture(  
        DataFetcherExceptionHandlerResult.newResult()  
            .error(error)  
            .build());  
}
```

3. Вопрос: какой `classification` будет у этой ошибки в ответе?

Задание 5. Включить GraphQL-метрики

1. Найдите в документации DGS, как включить трассировку запросов (подсказка: `TracingInstrumentation`).
2. Добавьте в `GraphQLSecurityConfig.java` новый `@Bean`:

`@Bean`



СОП

```
public Instrumentation tracingInstrumentation() {  
    return new TracingInstrumentation();  
}
```

1. Выполните любой запрос в GraphQL. В ответе появится поле "extensions": { "tracing": { ... } } с временем выполнения каждого резолвера.
2. Вопрос: почему трассировку не стоит включать на продакшене? Какие данные она раскрывает?



Асинхронная коммуникация через RabbitMQ

В предыдущих занятиях мы создали сервис demo-rest, предоставляющий REST и GraphQL API. Все взаимодействия были **синхронными**: клиент отправляет запрос, сервер обрабатывает и немедленно возвращает ответ. Теперь мы добавим **асинхронную коммуникацию** - при создании, обновлении или удалении книги/автора сервис будет публиковать **доменное событие** в брокер сообщений RabbitMQ, а отдельный audit-service будет принимать эти события и вести журнал аудита.

Ключевой архитектурный принцип: publisher **не знает**, кто получит его сообщение. Consumer **не знает**, кто отправил сообщение. Они связаны только через **контракт событий** и **брокер сообщений**.

1. Введение

В реальных системах микросервисы не могут общаться только синхронно. Представьте интернет-магазин:

1. **Оформление заказа** → нужно уведомить: склад (резервирование), платёжную систему (списание), почту (отправка письма), аналитику (статистика). Если делать это синхронно - один медленный сервис заблокирует всю цепочку.
2. **Аудит и комплаенс** - в финансовых системах каждое действие обязано фиксироваться в журнале. Аудит-сервис не должен влиять на скорость основных операций.

Событийная архитектура решает эти задачи через **асинхронность** (publisher не ждёт ответа), **декупленность** (сервисы не зависят друг от друга) и **масштабируемость** (добавить нового consumer'а можно без изменения publisher'а).

Необходимое ПО и настройка среды

Запуск RabbitMQ через Docker:

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:4-management
```

Параметр	Назначение
-d	Запуск в фоновом режиме (daemon)
--name rabbitmq	Имя контейнера для удобного управления



-p 5672:5672	Порт AMQP-протокола (для приложений)
-p 15672:15672	Порт Management UI (веб-интерфейс администрирования)
rabbitmq:4-management	Образ с включённым Management-плагином

Проверка - откройте <http://localhost:15672> и войдите с логином `guest` / паролем `guest`. Вы увидите панель управления RabbitMQ.

Сборка проекта:

```
.\mvnw install -DskipTests
```

Запуск (в двух отдельных терминалах):

Терминал 1 - publisher (REST + GraphQL API)

```
.\mvnw -pl demo-rest spring-boot:run -DskipTests
```

Терминал 2 - consumer (аудит-сервис)

```
.\mvnw -pl audit-service spring-boot:run -DskipTests
```

- **REST API:** <http://localhost:8080/swagger-ui.html>
- **Audit API:** <http://localhost:8081/api/audit>
- **RabbitMQ Management:** <http://localhost:15672>

2. Теория

2.1 Синхронная vs асинхронная коммуникация

Синхронная (REST, GraphQL): клиент отправляет запрос → ждёт → получает ответ.

Прямая зависимость - если сервис-получатель недоступен, операция провалится.

Асинхронная (RabbitMQ, Kafka): publisher отправляет сообщение в брокер → продолжает работу. Consumer получит сообщение позже, возможно через секунды.

2.2 Архитектура RabbitMQ - основные концепции

RabbitMQ - это **брокер сообщений**, реализующий протокол AMQP (Advanced Message Queuing Protocol). Его архитектура строится на четырёх ключевых компонентах - Producer → Exchange → Binding → Queue → Consumer.



Exchange (точка обмена)

Exchange - это «почтовое отделение». Publisher отправляет сообщение **не в очередь напрямую**, а в exchange с определённым **routing key**. Exchange решает, в какие очереди направить сообщение.

Типы exchange:

Тип	Routing key	Пример использования
Direct	Точное совпадение: book.created → book.created	Отправка в конкретную очередь
Topic	Паттерн: book.* → book.created, book.deleted	Гибкая маршрутизация по категориям
Fanout	Игнорируется - все очереди получают сообщение	Broadcast (уведомления, логирование)
Headers	По заголовкам сообщения	Сложная маршрутизация

В нашем проекте мы используем **Topic Exchange** - самый гибкий тип. Паттерны:

- ***** - **заменяет одно слово**: book.* → book.created, book.deleted, но не book.author.created
- **#** - **заменяет ноль или более слов**: # → все сообщения; book.# → book.created, book.author.created

Queue (очередь)

Queue - это буфер, хранящий сообщения до момента, когда consumer их прочитает.

Свойства:

- **Durable** - переживает перезапуск RabbitMQ (сохраняется на диск)
- **Exclusive** - принадлежит одному соединению, удаляется при отключении
- **Auto-delete** - удаляется, когда последний consumer отключается

Binding (привязка)

Binding - правило, связывающее exchange с queue. Указывает **binding key** - паттерн, по которому фильтруются сообщения:

```
// Все события (#) из exchange "books.events" → в очередь "q.audit.events"
bind(auditQueue).to(eventsExchange).with("#");
```



```
// Только события книг → в очередь "q.books.notifications"  
bind(booksQueue).to(eventsExchange).with("book.*");
```

```
// Только создание авторов → в очередь "q.authors.welcome"  
bind(welcomeQueue).to(eventsExchange).with("author.created");
```

Routing Key (ключ маршрутизации)

Routing key - строка, которую publisher прикрепляет к каждому сообщению. Exchange сравнивает routing key с binding key всех привязанных очередей и доставляет сообщение в те, где произошло совпадение.

2.3 Контракт событий - модуль books-events-contract

Аналогично REST API (books-api-contract), мы создаём **отдельный модуль** (books-events-contract) для описания событий. И publisher, и consumer зависят от этого контракта - если формат события изменится, оба модуля узнают об этом при компиляции.

Контракт - чистые Java records и sealed interfaces без фреймворков.

EventMetadata - CloudEvents-lite

В промышленных системах каждое событие сопровождается метаданными (спецификация [CloudEvents](#)). Мы используем упрощённую версию:

```
public record EventMetadata(  
    String eventId,        // UUID - для дедупликации  
    Instant timestamp,     // когда произошло  
    String source,         // кто отправил ("demo-rest")  
    String eventType       // тип ("book.created")  
) {  
    public static EventMetadata create(String source, String  
eventType) {  
        return new EventMetadata(  
            UUID.randomUUID().toString(),  
            Instant.now(),  
            source,  
            eventType  
        );  
    }  
}
```



}

}

- **eventId** - уникальный идентификатор события. Позволяет consumer'у обнаружить повторную доставку и не обрабатывать дубликат.
- **timestamp** - момент создания. Разница с **receivedAt** на стороне consumer'a показывает задержку доставки.
- **source** - имя сервиса-отправителя. В системе с десятками микросервисов критически важно знать, откуда пришло событие.
- **eventType** - совпадает с **routing key**. Позволяет определить тип без десериализации **payload**.

EventEnvelope - паттерн «конверт»

```
public record EventEnvelope<T>(
    EventMetadata metadata,
    T payload
) {
    public static <T> EventEnvelope<T> wrap(T payload, String
source, String eventType) {
        return new EventEnvelope<>(
            EventMetadata.create(source, eventType),
            payload
        );
    }
}
```

Аналогия с бумажной почтой: **metadata** - это конверт с адресом и штампом, **payload** - письмо внутри. Почтальону (брокеру) не нужно читать письмо, чтобы доставить его. Инфраструктурный код (логирование, дедупликация) работает с метаданными, не зная ничего о бизнес-содержимом.

BookEvent, AuthorEvent - sealed interfaces

```
public sealed interface BookEvent {
```

```
    record Created(
```

```
        Long bookId, String title, String isbn,
```



```
        Long authorId, String authorFullName,  
        String genre, Integer publishedYear  
    ) implements BookEvent {}
```

```
record Updated(  
    Long bookId, String title, String isbn,  
    String genre, Integer publishedYear  
    ) implements BookEvent {}
```

```
record Deleted(  
    Long bookId, String title  
    ) implements BookEvent {}
```

```
}
```

Sealed interface (Java 17+) - ограничивает набор наследников. Компилятор **гарантирует**, что BookEvent может быть **только** Created, Updated или Deleted. Это даёт:

- **Exhaustive switch** - компилятор проверяет, что обработаны все варианты
- **Безопасность** - невозможно создать «незнакомый» тип события
- **Документация** - набор событий виден из одного файла

RoutingKeys - общие константы

```
public final class RoutingKeys {  
    public static final String EXCHANGE = "books.events";  
  
    public static final String BOOK_CREATED = "book.created";  
    public static final String BOOK_UPDATED = "book.updated";  
    public static final String BOOK_DELETED = "book.deleted";  
  
    public static final String AUTHOR_CREATED = "author.created";  
    public static final String AUTHOR_DELETED = "author.deleted";  
  
    public static final String ALL_BOOK_EVENTS = "book.*";  
    public static final String ALL_EVENTS = "#";  
}
```




Вынесены в контракт, чтобы publisher и consumer **гарантированно использовали одни и те же строки**. Рассогласование routing key - частая ошибка: publisher отправляет с book.create, а consumer слушает book.created - и сообщение просто теряется без каких-либо ошибок.

2.4 Publisher - публикация событий из demo-rest

Конфигурация RabbitMQ (RabbitMQConfig.java - publisher)

@Configuration

```
public class RabbitMQConfig {

    @Bean
    public MessageConverter jsonMessageConverter(JsonMapper
jsonMapper) {
        return new JacksonJsonMessageConverter(jsonMapper);
    }

    @Bean
    public RabbitTemplate rabbitTemplate(ConnectionFactory
connectionFactory,
                                     MessageConverter
jsonMessageConverter) {
        RabbitTemplate template = new
RabbitTemplate(connectionFactory);
        template.setMessageConverter(jsonMessageConverter);
        return template;
    }

    @Bean
    public TopicExchange eventsExchange() {
        return ExchangeBuilder
            .topicExchange(RoutingKeys.EXCHANGE)
            .durable(true)
            .build();
    }
}
```



```
}  
}
```

Publisher определяет только exchange - ему не нужно знать, сколько очередей подписано и какие consumer'ы их слушают. Промышленный принцип: producer и consumer **развязаны**.

BookEventPublisher - паттерн fire-and-forget

@Component

```
public class BookEventPublisher {  
  
    private final RabbitTemplate rabbitTemplate;  
  
    public void publishCreated(BookResponse book) {  
        var event = new BookEvent.Created(  
            book.getId(), book.getTitle(), book.getIsbn(),  
            book.getAuthor().getId(),  
book.getAuthor().getFullName(),  
            book.getGenre(), book.getPublishedYear()  
        );  
        send(RoutingKeys.BOOK_CREATED, event);  
    }  
  
    private void send(String routingKey, BookEvent event) {  
        try {  
            EventEnvelope<BookEvent> envelope =  
EventEnvelope.wrap(event, "demo-rest", routingKey);  
            rabbitTemplate.convertAndSend(RoutingKeys.EXCHANGE,  
routingKey, envelope);  
        } catch (Exception e) {  
            log.error("Не удалось отправить событие {}: {}",  
routingKey, e.getMessage());  
        }  
    }  
}
```



```
}
```

convertAndSend - три аргумента:

1. **Exchange name** - куда отправить
2. **Routing key** - как маршрутизировать
3. **Object** - что отправить (Jackson сериализует в JSON)

try-catch - **критически важно**. Если RabbitMQ недоступен, событие потеряется, но бизнес-операция (создание книги) **не откатится**. Это сознательный компромисс: потеря аудит-записи допустима, отказ в создании книги - нет.

Интеграция в BookService

@Service

```
public class BookService {  
  
    private final BookEventPublisher eventPublisher;  
  
    public BookResponse createBook(BookRequest request) {  
        // ... создаём книгу ...  
        storage.books.put(id, book);  
  
        // Публикуем событие ПОСЛЕ успешного сохранения  
        eventPublisher.publishCreated(book);  
  
        return book;  
    }  
}
```

Событие публикуется **после** успешного завершения операции. Если публикация упадёт - книга уже создана, событие потеряется (допустимая потеря).

2.5 Consumer - audit-service

Конфигурация RabbitMQ (RabbitMQConfig.java - consumer)

Consumer определяет **полную топологию**: exchange, очереди, привязки и Dead Letter Queue:

@Configuration



```
public class RabbitMQConfig {

    public static final String AUDIT_QUEUE = "q.audit.events";
    public static final String AUDIT_DLQ = "q.audit.events.dlq";

    // JSON-конвертер
    @Bean
    public      MessageConverter      jsonMessageConverter(JsonMapper
jsonMapper) {
        return new JacksonJsonMessageConverter(jsonMapper);
    }

    // Listener Container Factory
    @Bean
    public      SimpleRabbitListenerContainerFactory
rabbitListenerContainerFactory(
        ConnectionFactory connectionFactory,
        MessageConverter jsonMessageConverter) {
        SimpleRabbitListenerContainerFactory  factory  =  new
SimpleRabbitListenerContainerFactory();
        factory.setConnectionFactory(connectionFactory);
        factory.setMessageConverter(jsonMessageConverter);
        factory.setConcurrentConsumers(1);          // 1 поток
(учебная среда)
        factory.setMaxConcurrentConsumers(3);      // максимум 3
        factory.setDefaultRequeueRejected(false); // при ошибке
в DLQ, не возвращать в очередь
        return factory;
    }

    // Topic Exchange
    @Bean
```



```
public TopicExchange eventsExchange() {
    return
ExchangeBuilder.topicExchange(RoutingKeys.EXCHANGE).durable(true).build();
}

// Dead Letter Exchange
@Bean
public DirectExchange deadLetterExchange() {
    return
ExchangeBuilder.directExchange(RoutingKeys.EXCHANGE
".dlx").durable(true).build();
}

// Основная очередь с перенаправлением в DLQ
@Bean
public Queue auditQueue() {
    return QueueBuilder.durable(AUDIT_QUEUE)
        .deadLetterExchange(RoutingKeys.EXCHANGE
".dlx")
        .deadLetterRoutingKey(AUDIT_DLQ)
        .build();
}

// Dead Letter Queue
@Bean
public Queue deadLetterQueue() {
    return QueueBuilder.durable(AUDIT_DLQ).build();
}

// Привязки
@Bean
```



```
        public Binding auditBinding(Queue auditQueue, TopicExchange
eventsExchange) {
            return
BindingBuilder.bind(auditQueue).to(eventsExchange).with(RoutingKeys.AL
L_EVENTS);
        }
```

```
        @Bean
        public Binding dlqBinding(Queue deadLetterQueue,
DirectExchange deadLetterExchange) {
            return
BindingBuilder.bind(deadLetterQueue).to(deadLetterExchange).with(AUDIT
_DLQ);
        }
    }
```

Dead Letter Queue (DLQ) - «мёртвая очередь». Сообщения попадают в DLQ, когда:

- **Consumer выбросил исключение и `requeue=false`**
- **TTL сообщения истёк**
- **Основная очередь переполнена**

Без DLQ «битые» сообщения просто теряются. DLQ сохраняет их для расследования.

AuditEventListener - обработка событий

@Component

```
public class AuditEventListener {
```

```
    private final AuditStorage auditStorage;
    private final ObjectMapper jsonMapper;
```

```
    @RabbitListener(queues = "q.audit.events", messageConverter =
    "")
```

```
    public void handleEvent(Message message) {
        try {
            byte[] body = message.getBody();
```



```
JsonNode root = jsonMapper.readTree(body);

// 1. Извлекаем метаданные
JsonNode metaNode = root.get("metadata");
EventMetadata metadata =
jsonMapper.treeToValue(metaNode, EventMetadata.class);

// 2. Дедупликация (idempotent consumer)
if (auditStorage.isDuplicate(metadata.eventId())) {
    log.warn("Дубликат пропущен: eventId={}",
metadata.eventId());
    return;
}

// 3. Десериализуем payload по eventType
JsonNode payloadNode = root.get("payload");
String description =
buildDescription(metadata.eventType(), payloadNode);

// 4. Сохраняем аудит-запись
AuditEntry entry = auditStorage.save(new AuditEntry(
    0, metadata.eventId(), metadata.eventType(),
    metadata.source(), metadata.timestamp(),
    Instant.now(), description));

log.info("[AUDIT #{}] {} | {}",
entry.sequenceNumber(),
    metadata.eventType(), description);

} catch (Exception e) {
    throw new RuntimeException("Не удалось обработать
событие", e);
}
```



```
    }  
  }  
}
```

Почему ручная десериализация?

EventEnvelope<T> - generic тип. Jackson 3 не может автоматически определить, какой конкретный record (BookEvent.Created? AuthorEvent.Deleted?) подставить вместо T. Поэтому:

1. Парсим JSON в дерево JsonNode (без привязки к типу)
2. Читаем eventType из метаданных
3. По eventType десериализуем payload в конкретный record

Idempotent Consumer - защита от повторов

@Component

```
public class AuditStorage {
```

```
    private final Set<String> processedEventIds =  
    ConcurrentHashMap.newKeySet();
```

```
    public boolean isDuplicate(String eventId) {  
        return !processedEventIds.add(eventId);  
    }  
}
```

RabbitMQ гарантирует **at least once** доставку, но **не** exactly once. Повторная доставка возможна при перезапуске consumer'a до отправки ACK. Idempotent consumer - стандартный паттерн: перед обработкой проверяем eventId, если уже видели - пропускаем.

2.6 Конфигурация приложений

```
demo-rest (application.properties)
```

```
spring.rabbitmq.host=localhost
```

```
spring.rabbitmq.port=5672
```

```
spring.rabbitmq.username=guest
```

```
spring.rabbitmq.password=guest
```




```
audit-service (application.yml)

spring:
  application:
    name: audit-service
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest
    listener:
      simple:
        retry:
          enabled: true
          initial-interval: 2000      # первая попытка через 2 сек
          max-attempts: 3             # максимум 3 попытки
          multiplier: 2.0             # каждая следующая x2

server:
  port: 8081                        # не конфликтует с demo-rest (8080)
```

Retry - при ошибке обработки сообщение не теряется сразу, а повторяется 3 раза с экспоненциальным backoff (2с → 4с → 8с). После исчерпания попыток в DLQ.

3. Практический разбор

Убедитесь, что RabbitMQ, demo-rest (порт 8080) и audit-service (порт 8081) запущены.

Шаг 1. Проверяем RabbitMQ Management UI

Откройте <http://localhost:15672> (guest/guest).

Перейдите на вкладку **Exchanges** - вы увидите exchange books.events (тип: topic).

Перейдите на **Queues and Streams** - увидите:

- **q.audit.events** - основная очередь (0 сообщений, 2 consumer'а)
- **q.audit.events.dlq** - Dead Letter Queue (0 сообщений)



Шаг 2. Создаём автора и наблюдаем событие

Отправляем запрос `http://localhost:8080/swagger-ui/index.html`:

POST `http://localhost:8080/api/authors`

Content-Type: `application/json`

```
{
  "firstName": "Anton",
  "lastName": "Chekhov",
  "nationality": "Russian"
}
```

Что происходит:

1. **`AuthService.create()` сохраняет автора**
2. **`AuthorEventPublisher.publishCreated()` отправляет `author.created` в RabbitMQ**
3. **В логах `demo-rest`: Событие отправлено: `author.created [eventId=...]`**
4. В логах `audit-service`: `[AUDIT #1] author.created | Создан автор «Anton Chekhov»`

(национальность: Russian)

Проверяем аудит-лог:

GET `http://localhost:8081/api/audit`

Ответ:

```
{
  "totalEntries": 1,
  "showing": 1,
  "entries": [
    {
      "sequenceNumber": 1,
      "eventId": "58083155-d562-484b-a138-f35e6b50ad31",
      "eventType": "author.created",
      "source": "demo-rest",
      "eventTimestamp": "2026-04-11T10:56:02.780Z",
      "receivedAt": "2026-04-11T10:56:02.945Z",
    }
  ]
}
```



```
        "description": "Создан автор «Anton Chekhov»  
(национальность: Russian)"  
    }  
]  
}
```

Обратите внимание на **задержку доставки**: разница между eventTimestamp и receivedAt ~165 мс - столько заняла доставка через RabbitMQ.

Шаг 3. Создаём книгу

POST <http://localhost:8080/api/books>

Content-Type: application/json

```
{  
    "title": "The Cherry Orchard",  
    "isbn": "978-5-389-01234-5",  
    "authorId": 1,  
    "genre": "Drama",  
    "publishedYear": 1904  
}
```

В аудит-логе появится вторая запись: book.created | Создана книга «The Cherry Orchard» (ISBN: 978-5-389-01234-5).

Шаг 4. Удаляем автора (каскадное удаление)

DELETE <http://localhost:8080/api/authors/1>

В аудит-логе: author.deleted | Удалён автор «Лев Толстой» (удалено книг: 3).

Событие author.deleted содержит количество каскадно удалённых книг - эта информация фиксируется **до** удаления из хранилища.

Шаг 5. Проверяем топологию в RabbitMQ Management

Откройте <http://localhost:15672> → **Exchanges** → books.events → вкладка **Bindings**:

Binding # означает - очередь получает **все** сообщения из exchange. Если бы мы хотели получать только события книг, binding key был бы book.*.



Шаг 6. Наблюдаем JSON-сообщение

В RabbitMQ Management → **Queues** → q.audit.events → **Get Message(s)** (установите Ack mode: Nack message requeue true). Если в очереди есть сообщение, вы увидите его содержимое:

```
{
  "metadata": {
    "eventId": "5a4fa5fc-0dc9-4dc5-b2a5-9fed12e92368",
    "timestamp": "2026-04-11T10:56:27.010Z",
    "source": "demo-rest",
    "eventType": "book.created"
  },
  "payload": {
    "bookId": 4,
    "title": "The Cherry Orchard",
    "isbn": "978-5-389-01234-5",
    "authorId": 1,
    "authorFullName": "Лев Толстой",
    "genre": "Drama",
    "publishedYear": 1904
  }
}
```

Структура чётко разделена: metadata (инфраструктура) и payload (бизнес-данные).

5. Дополнительные материалы

Шпаргалка: Spring AMQP аннотации и классы

Элемент	Назначение
@RabbitListener(queues = "...")	Привязывает метод к AMQP-очереди
RabbitTemplate	Отправка сообщений (фасад)
JacksonJsonMessageConverter	JSON ↔ Java конвертер (Jackson 3)
TopicExchange	Exchange с маршрутизацией по паттернам



DirectExchange	Exchange с точным совпадением routing key
QueueBuilder.durable("...")	Создание durable-очереди
BindingBuilder.bind().to().with()	Привязка очереди к exchange
SimpleRabbitListenerContainerFactory	Фабрика для настройки consumer'ов
ConnectionFactory	Управление AMQP-соединениями

Шпаргалка: Docker-команды для RabbitMQ

Запуск

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672
rabbitmq:4-management
```

Остановка / Запуск

```
docker stop rabbitmq
docker start rabbitmq
```

Очистка очереди

```
docker exec rabbitmq rabbitmqctl purge_queue q.audit.events
docker exec rabbitmq rabbitmqctl purge_queue q.audit.events.dlq
```

Статус

```
docker          exec          rabbitmq          rabbitmqctl          status
```



gRPC и синхронная межсервисная коммуникация на Protocol Buffers

В предыдущей работе мы добавили асинхронную коммуникацию (RabbitMQ): demo-rest публикует события, audit-service слушает и ведёт аудит-лог. Теперь мы добавим синхронную межсервисную коммуникацию по протоколу gRPC – бинарный, строго типизированный RPC-фреймворк от Google.

Мы создадим два новых сервиса:

- **grpc-analytics-server** – gRPC-сервер, выполняющий «аналитику» книг (оценка времени чтения, сложности, эпохи)
- **grpc-enrichment-client** – «мост» между RabbitMQ и gRPC: слушает событие book.created из шины, вызывает gRPC-сервер, публикует обогащённое событие book.enriched обратно

Демонстрируется типовой случай event-triggered synchronous call – асинхронное событие инициирует синхронный вызов к другому сервису, результат снова уходит в шину.

1. Введение

В микросервисных архитектурах сервисы общаются друг с другом. REST удобен для внешних API, но для внутренней коммуникации между сервисами есть более эффективные протоколы:

1. **Рекомендательная система** – при создании книги нужно в реальном времени рассчитать рекомендательный балл. REST-вызов с JSON добавит накладные расходы на сериализацию текста. gRPC с бинарным protobuf работает в 5-10 раз быстрее и потребляет меньше трафика.
2. **Межсервисный обмен данными в высоконагруженных системах** – Google, Netflix, Uber используют gRPC для внутренних коммуникаций между десятками тысяч микросервисов, потому что HTTP/2 мультиплексирование позволяет отправлять сотни запросов по одному TCP-соединению.
3. **Строго типизированный контракт** – в отличие от REST, где URL и формат JSON проверяются только в runtime, gRPC-контракт проверяется при компиляции. Если сервер поменял формат ответа, клиент не скомпилируется.

Необходимое ПО и настройка среды

Требования:

- Java 21, Maven 3.9+, IDE с поддержкой Maven
- Docker (для запуска RabbitMQ)



- Все предыдущие занятия выполнены (проект собирается, `.\mvnw install -DskipTests` проходит)
- RabbitMQ запущен: `docker start rabbitmq` (или `docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:4-management`)

Сборка проекта:

```
.\mvnw install -DskipTests
```

Все 7 модулей должны собраться.

Запуск (в четырёх отдельных терминалах):

Терминал 1 – REST + GraphQL + event publisher

```
.\mvnw spring-boot:run -pl demo-rest -DskipTests
```

Терминал 2 – аудит-сервис (consumer событий)

```
.\mvnw spring-boot:run -pl audit-service -DskipTests
```

Терминал 3 – gRPC-сервер аналитики (порт gRPC: 9090, HTTP: 8083)

```
.\mvnw spring-boot:run -pl grpc-analytics-server -DskipTests
```

Терминал 4 – gRPC-клиент обогащения (RabbitMQ → gRPC → RabbitMQ, HTTP: 8082)

```
.\mvnw spring-boot:run -pl grpc-enrichment-client -DskipTests
```

2. Теория

2.1 Что такое gRPC?

gRPC (Google Remote Procedure Call) – фреймворк для удалённого вызова процедур.

Клиент вызывает метод сервера так, как если бы он был локальным:

```
// Клиент вызывает удалённый метод – выглядит как обычный вызов
BookAnalysisResponse response =
analyticsStub.analyzeBook(request);
```

Под капотом gRPC:

1. Сериализует request в бинарный формат (Protocol Buffers)
2. Отправляет по HTTP/2 на сервер



3. Сервер десериализует, выполняет логику, сериализует ответ

4. Клиент получает ответ и десериализует в Java-объект

2.2 Protocol Buffers (Protobuf)

Protocol Buffers – язык описания данных и сериализации от Google. Аналог JSON/XML, но бинарный – данные занимают меньше места и парсятся быстрее.

Файл .proto описывает:

- **message** – структуры данных (аналог DTO/record в Java)
- **service** – набор RPC-методов (аналог интерфейса контроллера)

Синтаксис сообщений (message)

```
syntax = "proto3"; // версия protobuf
```

```
option java_package = "edu.rutmiit.demo.grpc"; // Java-пакет для генерации
```

```
option java_multiple_files = true; // каждый класс в отдельном файле
```

```
// Запрос – аналог Java record
```

```
message AnalyzeBookRequest {  
    int64 book_id      = 1; // номер поля (не значение!)  
    string title       = 2;  
    string genre       = 3;  
    int32 published_year = 4;  
    string language    = 5;  
    string author_name  = 6;  
}
```

Номера полей (= 1, = 2, ...) – это **идентификаторы** в бинарном формате, а не значения по умолчанию. Правила:

- **Номера нельзя менять после релиза (backward compatibility)**
- **Можно добавлять новые поля с новыми номерами**
- **Удалённые номера нельзя переиспользовать**

Типы данных:



Protobuf	Java	Описание
int32	int	32-бит целое
int64	long	64-бит целое
double	double	Число с плавающей точкой
string	String	Строка (UTF-8)
bool	boolean	Логическое значение
bytes	ByteString	Байтовый массив
repeated	List<T>	Коллекция

Синтаксис сервиса (service)

```
service BookAnalytics {  
    // Unary RPC - один запрос это один ответ  
    rpc AnalyzeBook (AnalyzeBookRequest) returns  
    (BookAnalysisResponse);  
}
```

Типы RPC:

Тип	Синтаксис	Описание
Unary	rpc Method(Req) returns (Resp)	Один запрос это один ответ (как REST)
Server streaming	rpc Method(Req) returns (stream Resp)	Один запрос это поток ответов
Client streaming	rpc Method(stream Req) returns (Resp)	Поток запросов это один ответ
Bidirectional	rpc Method(stream Req) returns (stream Resp)	Поток запросов это поток ответов

В нашем проекте мы используем **unary RPC** – самый простой и распространённый тип.

2.3 Генерация Java-кода из .proto

Компилятор protoc + плагин protoc-gen-grpc-java генерируют из .proto файла:



1. Классы сообщений – `AnalyzeBookRequest`, `BookAnalysisResponse` (immutable объекты с Builder-паттерном)
2. Базовый класс сервера – `BookAnalyticsGrpc.BookAnalyticsImplBase` (наследуем и реализуем логику)
3. Клиентские стабы – `BookAnalyticsGrpc.BookAnalyticsBlockingStub` (синхронный клиент)

В Maven это настраивается через `protobuf-maven-plugin`:

```
<plugin>
  <groupId>org.xolstice.maven.plugins</groupId>
  <artifactId>protobuf-maven-plugin</artifactId>
  <version>0.6.1</version>
  <configuration>
    <!-- Бинарник protoc (компилятор proto → Java) -->
    <protocArtifact>com.google.protobuf:protoc:3.25.5:exe:${os.detected.classifier}</protocArtifact>
    <!-- Плагин для генерации gRPC-стабов -->
    <pluginId>grpc-java</pluginId>
    <pluginArtifact>io.grpc:protoc-gen-grpc-java:1.66.0:exe:${os.detected.classifier}</pluginArtifact>
  </configuration>
  <executions>
    <execution>
      <goals>
        <goal>compile</goal> <!-- сообщения (message) -->
        <goal>compile-custom</goal> <!-- стабы (service) -->
      </goals>
    </execution>
  </executions>
</plugin>
```



`${os.detected.classifier}` – автоматически определяемая платформа (windows-x86_64, linux-x86_64, osx-aarch_64). Для этого нужен `os-maven-plugin` в секции `<extensions>`.

2.4 gRPC-контракт – модуль `books-grpc-contract`

Аналогично `books-api-contract` (REST) и `books-events-contract` (события), gRPC-контракт выделен в отдельный Maven-модуль.

Зависимости: `grpc-protobuf`, `grpc-stub`, `protobuf-java`, `javax.annotation-api`.

Полный контракт (`book_analytics.proto`):

```
syntax = "proto3";
```

```
option java_package = "edu.rutmiit.demo.grpc";
```

```
option java_multiple_files = true;
```

```
package bookanalytics;
```

```
// Сервис аналитики книг – unary RPC
```

```
service BookAnalytics {
```

```
    rpc      AnalyzeBook      (AnalyzeBookRequest)      returns  
(BookAnalysisResponse);  
}
```

```
// Запрос
```

```
message AnalyzeBookRequest {
```

```
    int64  book_id           = 1;
```

```
    string title             = 2;
```

```
    string genre             = 3;
```

```
    int32  published_year    = 4;
```

```
    string language          = 5;
```

```
    string author_name       = 6;
```

```
}
```

```
// Ответ
```

```
message BookAnalysisResponse {
```



```
int64 book_id = 1;
int32 estimated_reading_minutes = 2;
string difficulty_level = 3;
double recommendation_score = 4;
string era_classification = 5;
}
```

2.5 gRPC-сервер – grpc-analytics-server

Реализация сервиса (BookAnalyticsServiceImpl)

Наследуем сгенерированный базовый класс – аналог implements AuthorApi в REST:

```
public class BookAnalyticsServiceImpl extends
BookAnalyticsGrpc.BookAnalyticsImplBase {

    @Override
    public void analyzeBook(AnalyzeBookRequest request,
                           StreamObserver<BookAnalysisResponse>
responseObserver) {

        // 1. Вычисляем метрики
        int readingMinutes =
estimateReadingTime(request.getGenre(), request.getPublishedYear());
        String difficulty =
classifyDifficulty(request.getGenre(), request.getPublishedYear());
        double score =
calculateRecommendationScore(request.getGenre(),
request.getPublishedYear());
        String era = classifyEra(request.getPublishedYear());

        // 2. Формируем ответ через Builder
        BookAnalysisResponse response =
BookAnalysisResponse.newBuilder()
            .setBookId(request.getBookId())
            .setEstimatedReadingMinutes(readingMinutes)
```



```
.setDifficultyLevel(difficulty)
.setRecommendationScore(score)
.setEraClassification(era)
.build();
```

```
// 3. Отправляем ответ и завершаем RPC
```

```
responseObserver.onNext(response);
responseObserver.onCompleted();
```

```
}
```

```
}
```

StreamObserver – callback-интерфейс gRPC. Для unary RPC всегда:

- **onNext(response)** – отправить ответ (вызывается один раз)
- **onCompleted()** – завершить RPC
- **onError(throwable)** – сообщить об ошибке (вместо onNext + onCompleted)

Protobuf Builder – объекты сообщений неизменяемы (immutable). Для создания используется паттерн Builder: `Message.newBuilder().setField(...).build()`.

Управление жизненным циклом (`GrpcServerLifecycle`)

gRPC-сервер – это **отдельный** сетевой сервер (не Tomcat). Он слушает свой порт (9090) и требует явного `start()` / `shutdown()`:

```
@Component
```

```
public class GrpcServerLifecycle implements SmartLifecycle {
```

```
    @Value("${grpc.server.port:9090}")
```

```
    private int grpcPort;
```

```
    private Server server;
```

```
    private boolean running = false;
```

```
    @Override
```

```
    public void start() {
```

```
        server = ServerBuilder.forPort(grpcPort)
```

```
            .addService(new BookAnalyticsServiceImpl())
```



```
        .build()
        .start();
        running = true;
    }

    @Override
    public void stop() {
        if (server != null) {
            server.shutdown();
            running = false;
        }
    }

    @Override
    public boolean isRunning() { return running; }

    @Override
    public int getPhase() { return Integer.MAX_VALUE; }
}
```

SmartLifecycle – Spring-интерфейс для компонентов с управляемым запуском/остановкой.

getPhase() = Integer.MAX_VALUE – gRPC-сервер стартует **последним** (после всех Spring-бинов).

```
ServerBuilder
ServerBuilder.forPort(9090) // на каком порту слушать
    .addService(serviceImpl) // зарегистрировать реализацию
сервиса
    .addService(anotherService) // можно несколько сервисов на
одном порту
    .build() // создать сервер (без запуска)
    .start(); // начать приём запросов
```



2.6 gRPC-клиент – grpc-enrichment-client

Конфигурация клиента (GrpcClientConfig)

@Configuration

```
public class GrpcClientConfig {
```

```
    @Value("${grpc.client.analytics-server.host:localhost}")
```

```
    private String grpcHost;
```

```
    @Value("${grpc.client.analytics-server.port:9090}")
```

```
    private int grpcPort;
```

```
    @Bean
```

```
    public ManagedChannel managedChannel() {
```

```
        return ManagedChannelBuilder
```

```
            .forAddress(grpcHost, grpcPort)
```

```
            .usePlaintext()           // без TLS – только для
```

```
разработки и нашей лабы
```

```
            .build();
```

```
    }
```

```
    @Bean
```

```
    public BookAnalyticsGrpc.BookAnalyticsBlockingStub
```

```
bookAnalyticsStub(ManagedChannel channel) {
```

```
    return BookAnalyticsGrpc.newBlockingStub(channel);
```

```
}
```

```
    @PreDestroy
```

```
    public void shutdown() {
```

```
        if (channel != null && !channel.isShutdown()) {
```

```
            channel.shutdown();
```

```
        }
```

```
    }
```



```
}
```

ManagedChannel – аналог HttpClient для REST. Ключевые свойства:

- **Управляет пулом TCP-соединений (HTTP/2 мультиплексирование)**
- **Один канал обслуживает множество одновременных RPC-вызовов**
- **Поддерживает автоматический reconnect при обрыве**
- **Требует shutdown() при завершении приложения (иначе утечка ресурсов)**

BlockingStub – синхронный клиент. Вызов метода **блокирует поток** до получения ответа. Для async используются FutureStub или Stub (callback).

usePlaintext() – отключает TLS. В продакшене – обязательно TLS с сертификатами.

Слушатель событий (BookCreatedListener)

Этот компонент – «мост» между асинхронной и синхронной коммуникацией:

@Component

```
public class BookCreatedListener {
```

```
    private final BookAnalyticsGrpc.BookAnalyticsBlockingStub  
analyticsStub;
```

```
    private final EnrichmentEventPublisher enrichmentPublisher;  
    private final ObjectMapper jsonMapper;
```

```
    @RabbitListener(queues = "q.enrichment.book-created",  
messageConverter = "")
```

```
    public void handleBookCreated(Message message) {
```

```
        // 1. Десериализуем событие book.created из JSON
```

```
        JsonNode root = jsonMapper.readTree(message.getBody());
```

```
        BookEvent.Created bookCreated = jsonMapper.treeToValue(  
            root.get("payload"), BookEvent.Created.class);
```

```
        // 2. Формируем gRPC-запрос
```

```
        AnalyzeBookRequest grpcRequest =
```

```
AnalyzeBookRequest.newBuilder()
```

```
    .setBookId(bookCreated.bookId())
```




```
                .setTitle(bookCreated.title())
                .setGenre(bookCreated.genre() != null ?
bookCreated.genre() : "")
                .setPublishedYear(bookCreated.publishedYear() !=
null
                                ? bookCreated.publishedYear() : 0)
                .build();

// 3. Вызываем gRPC-сервер (синхронно)
BookAnalysisResponse grpcResponse =
analyticsStub.analyzeBook(grpcRequest);

// 4. Публикуем обогащённое событие обратно в RabbitMQ
BookEvent.Enriched enriched = new BookEvent.Enriched(
    grpcResponse.getBookId(),
    bookCreated.title(),
    grpcResponse.getEstimatedReadingMinutes(),
    grpcResponse.getDifficultyLevel(),
    grpcResponse.getRecommendationScore(),
    grpcResponse.getEraClassification()
);
enrichmentPublisher.publishEnriched(enriched);
}
}
```

2.7 Расширение контракта событий

Для нового события `book.enriched` мы расширили `books-events-contract`:

BookEvent.java – новый record в sealed interface:

```
public sealed interface BookEvent {
    record Created(...) implements BookEvent {}
    record Updated(...) implements BookEvent {}
    record Deleted(...) implements BookEvent {}
}
```



// Результат gRPC-обогащения

```
record Enriched(  
    Long bookId,  
    String title,  
    int estimatedReadingMinutes,  
    String difficultyLevel,  
    double recommendationScore,  
    String eraClassification  
) implements BookEvent {}
```

RoutingKeys.java – новая константа:

```
public static final String BOOK_ENRICHED = "book.enriched";
```

3. Практический разбор

Убедитесь, что RabbitMQ запущен, и все 4 сервиса работают (demo-rest, audit-service, grpc-analytics-server, grpc-enrichment-client).

Шаг 1. Проверяем запуск gRPC-сервера

В логах grpc-analytics-server вы должны увидеть:

gRPC-сервер запущен на порту 9090

Сервис: BookAnalytics.AnalyzeBook()

Это означает, что gRPC-сервер принимает запросы. В отличие от REST, нет Swagger UI – gRPC сервер не отвечает на HTTP/1.1 запросы браузера.

Шаг 2. Проверяем очереди в RabbitMQ

Откройте <http://localhost:15672> (guest/guest) → **Queues and Streams**. Вы увидите новую очередь:

Очередь	Binding Key	Consumer
q.audit.events	# (все события)	audit-service
q.enrichment.book-created	book.created	grpc-enrichment-client

Обратите внимание: q.enrichment.book-created привязана **только** к book.created, а не ко всем событиям (#). Enrichment-клиенту не нужны author.created или book.deleted.



Шаг 3. Создаём автора

POST `http://localhost:8080/api/authors`

Content-Type: `application/json`

```
{
  "firstName": "Лев",
  "lastName": "Толстой",
  "nationality": "Russian"
}
```

В логах `audit-service`: `[AUDIT #1] author.created` | Создан автор «Лев Толстой»

В логах `grpc-enrichment-client` – **ничего**. Автор не вызывает обогащение, только книги.

Шаг 4. Создаём книгу и наблюдаем полную цепочку

POST `http://localhost:8080/api/books`

Content-Type: `application/json`

```
{
  "title": "Война и мир",
  "isbn": "978-5-389-06259-8",
  "authorId": 1,
  "genre": "Роман",
  "publishedYear": 1869
}
```

Что происходит (наблюдаем в 4 терминалах):

Терминал 1 (demo-rest):

Событие отправлено: `book.created [eventId=abc-123...]`

Терминал 4 (grpc-enrichment-client):

Получено событие `book.created: bookId=1, «Война и мир»`
`[eventId=abc-123...]`

Вызов gRPC: `BookAnalytics.AnalyzeBook(bookId=1)`

gRPC ответ получен: `bookId=1, время=936мин, сложность=CLASSIC, балл=9.0, эпоха=CLASSICAL`



Книга обогащена: `bookId=1`, «Война и мир» → `book.enriched`
отправлено

Событие отправлено: `book.enriched [bookId=1, eventId=def-456...]`

Терминал 3 (grpc-analytics-server):

gRPC запрос: анализ книги `id=1` «Война и мир» (жанр: Роман, год: 1869)

gRPC ответ: книга `id=1`, время чтения=936мин, сложность=CLASSIC, балл=9.0, эпоха=CLASSICAL

Терминал 2 (audit-service):

[AUDIT #2] `book.created` | Создана книга «Война и мир» (ISBN: 978-5-389-06259-8)

[AUDIT #3] `book.enriched` | Книга обогащена `id=1` «Война и мир»
(время чтения: 936мин, сложность: CLASSIC, балл: 9.0, эпоха: CLASSICAL)

Шаг 5. Проверяем аудит-лог

GET `http://localhost:8081/api/audit`

Вы увидите **3 записи**: `author.created`, `book.created`, `book.enriched`. Обратите внимание:

- **`book.created`** – source: "demo-rest" (создан REST API)
- **`book.enriched`** – source: "grpc-enrichment-client" (создан enrichment-клиентом)

Шаг 6. Создаём ещё одну книгу с другим жанром

POST `http://localhost:8080/api/books`

Content-Type: `application/json`

```
{
  "title": "Евгений Онегин",
  "isbn": "978-5-389-01111-1",
  "authorId": 1,
  "genre": "Поэма",
  "publishedYear": 1833
}
```



Обратите внимание на **другие метрики**: жанр «Поэма» даёт меньшее время чтения (234 мин vs 936 мин для «Романа»). gRPC-сервер вычисляет разные значения в зависимости от входных данных.

Шаг 7. Тестируем отказоустойчивость – выключаем gRPC-сервер

1. Остановите `grpc-analytics-server` (Ctrl+C в Терминале 3)
2. Создайте книгу через REST
3. В логах `grpc-enrichment-client` вы увидите:

gRPC ошибка при обогащении книги: `null (UNAVAILABLE)`

1. Сообщение уйдёт в DLQ после исчерпания `retry`-попыток
2. Запустите `grpc-analytics-server` обратно – накопленные сообщения из DLQ не обработаются автоматически (DLQ – для расследования, не для автоматического повтора)

*Книга при этом **создана** (REST вернул 201). gRPC-обогащение – дополнительная операция, её отказ не влияет на основной бизнес-процесс.*

5. Дополнительные материалы

Шпаргалка: Protobuf-типы к типам Java

Protobuf	Java	Значение по умолчанию
int32	int	0
int64	long	0L
float	float	0.0f
double	double	0.0
bool	boolean	false
string	String	"" (пустая строка)
bytes	ByteString	пустой
repeated T	List<T>	пустой список

gRPC Java API

Элемент	Назначение
---------	------------



ServerBuilder.forPort(port)	Создание gRPC-сервера
.addService(impl)	Регистрация реализации сервиса
ManagedChannelBuilder.forAddress(host, port)	Создание клиентского канала
.usePlaintext()	Отключить TLS
ServiceGrpc.newBlockingStub(channel)	Синхронный клиентский стаб
ServiceGrpc.newFutureStub(channel)	Async стаб (возвращает ListenableFuture)
StreamObserver.onNext(response)	Отправить ответ клиенту
StreamObserver.onCompleted()	Завершить RPC
StreamObserver.onError(throwable)	Ответить ошибкой
Message.newBuilder().setField().build()	Создание protobuf-объекта

Maven-команды

Собрать всё

```
.\mvnw install -DskipTests
```

Только gRPC-контракт (перегенерация стабов)

```
.\mvnw install -pl books-grpc-contract -DskipTests
```

Запуск конкретного модуля

```
.\mvnw spring-boot:run -pl grpc-analytics-server -DskipTests
```

```
.\mvnw spring-boot:run -pl grpc-enrichment-client -DskipTests
```

Важные замечания

- Порядок запуска важен для gRPC. `grpc-analytics-server` должен быть запущен до `grpc-enrichment-client`, иначе gRPC-вызовы завершатся ошибкой `UNAVAILABLE`.
- В проде используют `retry` с `backoff` и `health checks`.
- `RabbitMQ` должен быть запущен до `enrichment-client` и `audit-service`.
Проверьте: `docker ps | grep rabbitmq`.



WebSocket

В предыдущих занятиях мы построили полную цепочку обработки данных: REST API публикует событие в RabbitMQ, который его маршрутизирует в enrichment-client, который обогащает через gRPC и все это audit-service логирует. Но у нас нет **обратной связи** с пользователем - он не знает, что произошло после его REST-запроса, пока не выполнит GET /api/audit вручную. Теперь мы добавим **push-уведомления в реальном времени** через WebSocket – пятый транспорт в нашей архитектуре.

Мы создадим новый микросервис:

- **notification-service – слушает все доменные события из RabbitMQ и мгновенно рассылает их подключённым браузерам по WebSocket**

Ключевой паттерн тут это **event-to-push notification** – события из шины автоматически транслируются в push-уведомления для подключённых клиентов.

1. Введение

Во всех современных веб-приложениях пользователи ожидают **мгновенной реакции** на действия:

1. **Мессенджеры и чаты – новые сообщения появляются без перезагрузки страницы. WhatsApp, Telegram, Slack – все используют WebSocket для доставки сообщений в реальном времени.**
2. **Дашборды и мониторинг – Grafana, Kibana, RabbitMQ Management UI обновляют графики и метрики в реальном времени, потому что серверы push'ат данные по WebSocket.**
3. **Уведомления в веб-приложениях – GitHub отправляет уведомления о новых PR, CI-системы сообщают о результатах сборки, торговые платформы транслируют изменения цен – всё это push через WebSocket.**

В нашем проекте WebSocket решает конкретную задачу – пользователь создал книгу через REST, и через доли секунды в его браузере появляется уведомление.

Необходимое ПО и настройка среды

Сборка проекта:

```
.\mvnw install -DskipTests
```

Запуск (в пяти отдельных терминалах):

```
# Терминал 1 – REST + GraphQL + event publisher
```

```
.\mvnw spring-boot:run -pl demo-rest -DskipTests
```



Терминал 2 – аудит-сервис

```
.\mvnw spring-boot:run -pl audit-service -DskipTests
```

Терминал 3 – gRPC-сервер аналитики

```
.\mvnw spring-boot:run -pl grpc-analytics-server -DskipTests
```

Терминал 4 – gRPC-клиент обогащения

```
.\mvnw spring-boot:run -pl grpc-enrichment-client -DskipTests
```

Терминал 5 –notification-service (WebSocket push)

```
.\mvnw spring-boot:run -pl notification-service -DskipTests
```

Центр уведомлений: откройте в браузере <http://localhost:8084/>

2. Теория

2.1 HTTP: проблема однонаправленности

HTTP (1.0/1.1) – протокол «запрос-ответ»:

- Клиент инициирует запрос и сервер отвечает
- Сервер не может отправить данные клиенту без предварительного запроса
- Для «реального времени» клиент вынужден опрашивать сервер (polling)

Polling (опрос) – клиент периодически спрашивает: «Есть новые данные?»

Проблемы polling'a:

- Задержка, так как данные приходят с опозданием (до интервала опроса)
- Трафик: 90%+ запросов возвращают пустой ответ
- Нагрузка: N клиентов × M запросов/мин это огромная нагрузка на сервер

2.2 WebSocket это полнодуплексная связь

WebSocket (RFC 6455) – протокол, обеспечивающий **двустороннюю** (full-duplex) постоянную связь через **одно TCP-соединение**.



WebSocket-соединение начинается как **обычный HTTP-запрос**, который «апгрейдится» до WebSocket:

Клиент делает запрос серверу (HTTP Upgrade request):

```
GET /ws/notifications HTTP/1.1
Host: localhost:8084
Upgrade: websocket      // хочу переключиться на WebSocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZS... // случайный ключ
Sec-WebSocket-Version: 13      // версия протокола
```

Сервер → Клиент (HTTP 101):

```
HTTP/1.1 101 Switching Protocols // "переключаюсь!"
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBi... // подтверждение ключа
```

После 101 Switching Protocols TCP-соединение **не закрывается**, а переходит в режим WebSocket. Дальнейшие данные передаются в **бинарных фреймах** (не HTTP).

2.3 Нативный WebSocket

В нашем проекте – нативный WebSocket. Причины:

- **Максимальная простота и прозрачность**
- **Полный контроль над форматом сообщений (свой JSON)**
- **Нет зависимостей от дополнительных протоколов**
- **Все современные браузеры поддерживают WebSocket нативно**

2.4 Spring WebSocket

Конфигурация WebSocket endpoint

@Configuration

@EnableWebSocket // включает поддержку WebSocket в Spring

```
public class WebSocketConfig implements WebSocketConfigurer {
```

```
    private final NotificationWebSocketHandler
notificationHandler;
```



```
@Override
public void
registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
    registry
        .addHandler(notificationHandler, "/ws/notifications")
// endpoint
        .setAllowedOrigins("*");    // CORS – для разработки
    }
}

/ws/notifications – путь, по которому клиент подключается: new
WebSocket("ws://localhost:8084/ws/notifications").

WebSocket-обработчик (Handler)

Наследуем TextWebSocketHandler – базовый класс для обработки текстовых
WebSocket-сообщений:

@Component
public class NotificationWebSocketHandler extends
TextWebSocketHandler {

    // Потокбезопасный реестр активных сессий
    private final Set<WebSocketSession> sessions =
ConcurrentHashMap.newKeySet();

    @Override
    public void afterConnectionEstablished(WebSocketSession
session) {
        sessions.add(session);
        // клиент подключился – добавляем в реестр
    }

    @Override
```



```
public void afterConnectionClosed(WebSocketSession session,
CloseStatus status) {
    sessions.remove(session);
    // клиент отключился – удаляем из реестра
}

@Override
protected void handleTextMessage(WebSocketSession session,
TextMessage message) {
    // клиент прислал сообщение (например, ping)
}

@Override
public void handleTransportError(WebSocketSession session,
Throwable exception) {
    sessions.remove(session);
    // ошибка транспорта – удаляем сессию
}

// Отправить сообщение всем подключённым клиентам
public void broadcast(String json) {
    TextMessage msg = new TextMessage(json);
    for (WebSocketSession session : sessions) {
        if (session.isOpen()) {
            session.sendMessage(msg);
        }
    }
}
```

2.5 Потокобезопасность и почему ConcurrentHashMap

WebSocket-обработчик вызывается из **разных потоков**:

- Поток HTTP – при подключении/отключении клиента



- Поток RabbitMQ Listener – при broadcast'e из EventNotificationListener
- Поток heartbeat – при проверке active sessions

Если использовать обычный HashSet, при одновременном вызове add() и remove() из разных потоков возникнет ConcurrentModificationException.

// не потокобезопасно:

```
private final Set<WebSocketSession> sessions = new HashSet<>();
```

// потокобезопасно:

```
private final Set<WebSocketSession> sessions =  
ConcurrentHashMap.newKeySet();
```

ConcurrentHashMap.newKeySet() – Set на основе ConcurrentHashMap, безопасный для параллельного доступа.

2.6 Broadcast паттерн рассылки

Broadcast – отправка одного сообщения всем подключённым клиентам:

```
public void broadcast(String json) {  
    if (sessions.isEmpty()) return;  
  
    TextMessage message = new TextMessage(json);  
    for (WebSocketSession session : sessions) {  
        if (session.isOpen()) {  
            try {  
                session.sendMessage(message);  
            } catch (IOException e) {  
                sessions.remove(session); // «мёртвая» сессия –  
удаляем  
            }  
        } else {  
            sessions.remove(session);  
        }  
    }  
}
```

Ключевые принципы:



- Проверяем `session.isOpen()` перед отправкой
- Каждая отправка обернута в `try/catch` – один «мёртвый» клиент не блокирует остальных
- «Мёртвые» сессии автоматически удаляются из реестра

2.7 Интеграция RabbitMQ и WebSocket (event-to-push)

Notification-service объединяет два транспорта:

- Вход с RabbitMQ (`@RabbitListener`) – получает все доменные события
- Выход с WebSocket (`broadcast()`) – рассылает уведомления браузерам

`EventNotificationListener` – слушатель RabbitMQ:

```
@RabbitListener(queues = "q.notifications.all", messageConverter
= "")
public void handleEvent(Message message) {
    // Парсим метаданные (eventType, eventId, source, timestamp)
    JsonNode root = jsonMapper.readTree(message.getBody());
    EventMetadata metadata =
    jsonMapper.treeToValue(root.get("metadata"), EventMetadata.class);

    // Дедупликация по eventId
    if (!processedEventIds.add(metadata.eventId())) return;

    // Формируем человекочитаемое уведомление
    String title = buildTitle(metadata.eventType()); //
    "Новая книга"
    String description = buildDescription(...); //
    "Создана книга «Война и мир»"

    // Сериализуем в JSON для WebSocket
    String json = jsonMapper.writeValueAsString(notification);

    // Рассылаем всем подключённым браузерам
    websocketHandler.broadcast(json);
}
```



```
}
```

Дедупликация – RabbitMQ может повторно доставить сообщение (at-least-once delivery). Набор processedEventIds (на основе ConcurrentHashMap) гарантирует, что одно событие отправляется по WebSocket только один раз.

2.8 JavaScript-клиент и нативный WebSocket API

Браузер подключается через стандартный WebSocket API:

```
// Создание подключения
```

```
const ws = new WebSocket("ws://localhost:8084/ws/notifications");
```

```
// Соединение установлено
```

```
ws.onopen = () => {  
    console.log("Подключено!");  
};
```

```
// Получено сообщение от сервера
```

```
ws.onmessage = (event) => {  
    const data = JSON.parse(event.data);  
    console.log("Уведомление:", data.title, data.description);  
};
```

```
// Соединение закрыто
```

```
ws.onclose = (event) => {  
    console.log("Отключено:", event.code, event.reason);  
};
```

```
// Ошибка
```

```
ws.onerror = (error) => {  
    console.error("Ошибка WebSocket:", error);  
};
```

Автоматическое переподключение (exponential backoff):

```
let reconnectAttempt = 0;
```



```
function connect() {  
    const ws = new WebSocket(WS_URL);  
  
    ws.onopen = () => {  
        reconnectAttempt = 0; // сбрасываем счётчик  
    };  
  
    ws.onclose = () => {  
        // Увеличиваем задержку: 1с, 2с, 4с, 8с, ..., max 30с  
        const delay = Math.min(1000 * Math.pow(2,  
reconnectAttempt), 30000);  
        reconnectAttempt++;  
        setTimeout(connect, delay);  
    };  
}
```

Exponential backoff – паттерн, при котором интервал между попытками переподключения растёт экспоненциально: 1с → 2с → 4с → 8с → 16с → 30с (max). Защищает сервер от лавины переподключений при массовом отказе.

3. Практический разбор

Убедитесь, что RabbitMQ запущен, и все 5 сервисов работают (demo-rest, audit-service, grpc-analytics-server, grpc-enrichment-client, notification-service).

Шаг 1. Открываем Центр уведомлений

Откройте в браузере <http://localhost:8084/>. Вы увидите:

- **Заголовок «Центр уведомлений»**
- **Индикатор статуса: зелёная точка и текст «Подключено»**
- **Пустое состояние: «Нет уведомлений»**

В логах notification-service вы увидите:

WebSocket подключён: 0 (всего: 1)

Это значит, что ваш браузер подключился к WebSocket-серверу.



Шаг 2. Проверяем очереди в RabbitMQ

Откройте `http://localhost:15672` в **Queues and Streams**. Вы увидите новую очередь `q.notifications.all`:

Обратите внимание: `q.notifications.all` привязана к `#` – она получает **абсолютно все** события, как и `q.audit.events`.

Шаг 3. Создаём автора и наблюдаем уведомление

POST `http://localhost:8080/api/authors`

Content-Type: `application/json`

```
{
  "firstName": "Лев",
  "lastName": "Толстой",
  "nationality": "Russian"
}
```

Что происходит:

1. **demo-rest публикует `author.created` в RabbitMQ**
2. **notification-service получает событие из `q.notifications.all`**
3. **EventNotificationListener формирует уведомление:**
 1. **title: «Новый автор»**
 2. **description: «Создан автор «Лев Толстой» (национальность: Russian)»**
4. **NotificationWebSocketHandler.broadcast() отправляет JSON всем клиентам**
5. **В браузере появляется карточка уведомления с анимацией**

В логах `notification-service`:

```
[NOTIFY]  author.created | Создан автор «Лев Толстой»
(национальность: Russian) (клиентов: 1)
```

Шаг 4. Создаём книгу – два уведомления подряд

POST `http://localhost:8080/api/books`

Content-Type: `application/json`

```
{
```




```
"title": "Война и мир",  
"isbn": "978-5-389-06259-8",  
"authorId": 1,  
"genre": "Роман",  
"publishedYear": 1869  
}
```

В браузере появляются **два** уведомления (снизу вверх по времени):

1. **Новая книга – book.created (source: demo-rest)**
2. **Аналитика книги – book.enriched (source: grpc-enrichment-client)**

Это демонстрирует, что notification-service получает **все** события – и от demo-rest, и от grpc-enrichment-client.

Шаг 5. Тестируем индикатор подключения

1. **Остановите notification-service (Ctrl+C в Терминале 5)**
2. **В браузере статус меняется на красный: «Отключено»**
3. **Через секунду – жёлтый: «Переподключение через 1с...»**
4. **Попытки переподключения с нарастающей задержкой (1с, 2с, 4с, 8с, ...)**
5. **Запустите notification-service обратно**
6. **Статус автоматически станет зелёным: «Подключено»**

Это – **exponential backoff** в действии.

Шаг 6. Тестируем несколько вкладок

1. **Откройте <http://localhost:8084/> в трёх вкладках браузера**
2. **В логах: WebSocket подключён: ... (всего: 3)**
3. **Создайте ещё одну книгу через REST**
4. **Все три вкладки получают уведомление одновременно – это broadcast**

Шаг 7. Откройте лог подключения

В Центре уведомлений нажмите **«Показать лог подключения»**. Вы увидите хронологию:

```
[12:35:01] Подключение к ws://localhost:8084/ws/notifications...  
[12:35:01] WebSocket подключён
```



[12:35:01] Сервер: Подключено к Notification Service
(подключений: 1)

Это – отладочный инструмент, встроенный в клиент.

Важные замечания

- **notification-service не зависит от gRPC. Он слушает RabbitMQ и рассылает WebSocket-уведомления. Можно запускать без gRPC-сервисов – тогда придут только book.created, book.updated, book.deleted, author.created, author.deleted.**
- **WebSocket – постоянное соединение. В отличие от REST (запрос => ответ => закрытие), WebSocket держит TCP-соединение открытым. Это важно учитывать для масштабирования (file descriptors, memory).**
- **Exponential backoff обязателен. Если 1000 клиентов потеряют соединение одновременно и все попытаются переподключиться через 1 секунду – сервер рухнет под нагрузкой. Экспоненциальная задержка (1с, 2с, 4с, 8с, ...) распределяет нагрузку.**
- **Центр уведомлений – статическая страница. Файл index.html обслуживается встроенным Tomcat из src/main/resources/static/. Никакого фронтенд-фреймворка.**